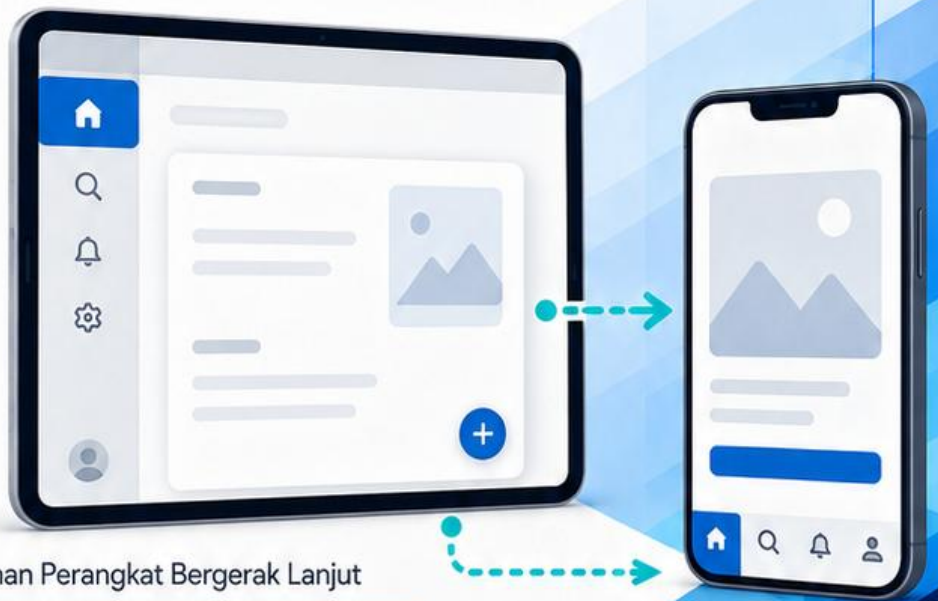


MODUL PRAKTIKUM 02

NAVIGASI DAN ROUTING PADA APLIKASI FLUTTER

Bottom App Bar • Navigation Bar • Drawer • Navigation Rail



Mata Kuliah

Pemrograman Perangkat Bergerak Lanjut

Modul

II – Navigasi dan Routing pada Aplikasi Flutter

Tahun

2026

DAFTAR ISI

A.	Tujuan Pembelajaran.....	3
B.	Tugas Pendahuluan	3
C.	Dasar Teori.....	3
a.	Konsep Navigator dan Route	4
b.	Tumpukan Navigasi (Navigation Stack)	4
c.	Named Routes	5
d.	Widget Navigasi pada Material Design 3	5
e.	Tab Navigation.....	5
f.	Navigasi Adaptif	6
D.	Alat dan Bahan.....	6
a.	Perangkat Keras (Hardware)	6
b.	Perangkat Lunak (Software)	6
E.	Materi Praktikum	6
a.	Menyiapkan Proyek Praktikum.....	6
b.	Konsep Navigator dan Routes.....	7
c.	Implementasi Navigasi Sederhana dengan Navigator.push() dan Navigator.pop()	9
d.	Navigasi dengan Named Routes	13
e.	Membuat Bottom Navigation Bar dengan NavigationBar.....	18
f.	Menggunakan BottomAppBar sebagai Wadah Aksi	23
g.	Membuat Navigation Drawer.....	26
h.	Membuat Navigation Rail untuk Layar Lebar	30
i.	Membuat Tab Navigation	34
j.	Ringkasan Perbedaan dan Panduan Memilih Navigasi.....	37
F.	Materi Praktikum dengan Prompt	38
a.	Menyiapkan Proyek Praktikum dengan Bantuan AI.....	39
b.	Memahami Navigator dan Routes dengan Bantuan AI.....	40
c.	Menerapkan Navigator.push dan Navigator.pop dengan Bantuan AI	41
d.	Menerapkan Named Routes dengan Bantuan AI	43
e.	Membuat NavigationBar dengan Bantuan AI.....	44
f.	Membuat BottomAppBar dengan Bantuan AI.....	46
g.	Membuat NavigationDrawer dengan Bantuan AI.....	47
h.	Membuat NavigationRail Adaptif dengan Bantuan AI.....	48
i.	Membuat Tab Navigation dengan Bantuan AI	49
j.	Prompt untuk Menelusuri Galat	50
k.	Menguji Aplikasi dan Menguji Pemahaman.....	51
G.	Tugas Praktikum	51
H.	Format Pengumpulan Tugas	54

A. Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Menjelaskan konsep Navigator, *route*, dan tumpukan navigasi (*navigation stack*) pada aplikasi Flutter.
- Membangun perpindahan halaman menggunakan `Navigator.push()` dan `Navigator.pop()` beserta `MaterialPageRoute`.
- Mengirim data ke halaman tujuan melalui konstruktor *widget* dan menerima data kembali melalui nilai kembalian `Navigator.pop()`.
- Menerapkan *named routes* melalui properti `initialRoute`, `routes`, dan `onGenerateRoute` pada `MaterialApp`.
- Membedakan penggunaan `Navigator.pushNamed()`, `pushReplacementNamed()`, `popUntil()`, dan `pushNamedAndRemoveUntil()`.
- Mengimplementasikan *widget* navigasi Material Design 3, yaitu `NavigationBar` (*bottom navigation bar*), `BottomAppBar`, `Drawer` atau `NavigationDrawer`, dan `NavigationRail`.
- Membuat *tab navigation* menggunakan `DefaultTabController`, `TabBar`, dan `TabBarView`.
- Menyusun navigasi adaptif yang berpindah otomatis antara `NavigationBar` dan `NavigationRail` berdasarkan lebar layar perangkat.
- Melakukan pengujian alur navigasi pada emulator, perangkat fisik, maupun peramban serta menelusuri galat yang berkaitan dengan konteks Navigator.

B. Tugas Pendahuluan

1. Apa yang dimaksud dengan Route dan Navigator pada Flutter, serta bagaimana keduanya bekerja sama dalam mengelola tumpukan (*stack*) halaman?
2. Jelaskan perbedaan antara `Navigator.push()`, `Navigator.pop()`, dan `Navigator.pushReplacement()` dalam mengelola perpindahan halaman pada aplikasi Flutter!
3. Apa yang dimaksud dengan *named routes* pada Flutter, dan jelaskan fungsi properti `initialRoute`, `routes`, serta `onGenerateRoute` pada `MaterialApp`!
4. Jelaskan perbedaan penggunaan `NavigationBar`, `BottomAppBar`, `NavigationDrawer`, dan `NavigationRail` sebagai *widget* navigasi pada Material Design 3!
5. Apa yang dimaksud dengan navigasi adaptif pada Flutter, dan jelaskan bagaimana lebar layar memengaruhi pemilihan *widget* navigasi yang digunakan!
6. Jelaskan perbedaan mendasar antara *tab navigation* (menggunakan `TabBar`, `TabBarView`, dan `TabController`) dengan `Navigator.push()` dalam mengelola perpindahan konten pada aplikasi Flutter!

C. Dasar Teori

Aplikasi perangkat bergerak pada umumnya terdiri atas banyak halaman. Pengguna berpindah dari daftar menuju rincian, dari rincian menuju formulir, kemudian kembali lagi ke halaman sebelumnya. Flutter menyediakan mekanisme navigasi yang terpadu untuk mengatur perpindahan tersebut sekaligus menjaga riwayat halaman agar tombol kembali pada perangkat tetap berperilaku wajar.

a. Konsep Navigator dan Route

Route adalah abstraksi satu layar atau satu halaman pada Flutter. Sebuah *route* tidak harus berupa halaman penuh, karena dialog dan lembar bawah (*bottom sheet*) juga diperlakukan sebagai *route*. Navigator adalah *widget* yang mengelola sekumpulan *route* dalam bentuk tumpukan (*stack*). Navigator bertugas menampilkan *route* yang berada pada posisi paling atas tumpukan serta menyimpan *route* di bawahnya sebagai riwayat.

Ketika `MaterialApp` digunakan, Flutter secara otomatis menyisipkan sebuah Navigator pada akar aplikasi. Oleh karena itu, seluruh *widget* yang berada di bawah `MaterialApp` dapat memperoleh Navigator terdekat melalui pemanggilan `Navigator.of(context)`. Konteks (`BuildContext`) yang digunakan harus berada di bawah `MaterialApp`, sebab konteks inilah yang dipakai untuk menelusuri Navigator pada pohon *widget*.

`MaterialPageRoute` merupakan implementasi *route* yang paling sering digunakan pada aplikasi berbasis Material Design. *Route* jenis ini menyediakan animasi transisi yang sesuai dengan kebiasaan masing-masing platform, yaitu transisi geser ke samping pada iOS dan transisi naik disertai pudar pada Android.

b. Tumpukan Navigasi (Navigation Stack)

Navigator bekerja dengan prinsip *last in first out*. Halaman yang paling akhir ditambahkan adalah halaman yang paling dahulu dilepas. Pemahaman terhadap perilaku tumpukan ini penting karena banyak kesalahan navigasi disebabkan oleh tumpukan yang terus bertambah tanpa pernah dilepas.

- Operasi `push` menambahkan *route* baru pada puncak tumpukan sehingga halaman baru tampil di atas halaman sebelumnya.
- Operasi `pop` melepas *route* pada puncak tumpukan sehingga aplikasi kembali ke halaman sebelumnya.
- Operasi `pushReplacement` menambahkan *route* baru sekaligus melepas *route* yang sedang aktif, sehingga halaman sebelumnya tidak dapat dijangkau kembali dengan tombol kembali. Pola ini lazim digunakan pada perpindahan dari halaman masuk (login) menuju halaman utama.
- Operasi `popUntil` melepas *route* secara berulang sampai syarat tertentu terpenuhi, misalnya sampai halaman utama kembali berada pada puncak tumpukan.
- Pemeriksaan `canPop` dapat digunakan untuk mengetahui apakah masih tersedia *route* di bawah *route* aktif sebelum operasi `pop` dijalankan.



Gambar 2.1 Ilustrasi tumpukan (*stack*) *route* yang dikelola oleh Navigator

c. Named Routes

Pada aplikasi berskala kecil, penulisan `MaterialPageRoute` secara langsung di setiap tempat masih dapat dikelola. Namun ketika jumlah halaman bertambah, pendekatan tersebut menyebabkan nama kelas halaman tersebar pada banyak berkas. *Named routes* menyelesaikan persoalan tersebut dengan cara mendaftarkan setiap halaman pada satu tempat, yaitu properti *routes* milik `MaterialApp`, kemudian memanggilnya melalui nama berupa teks.

- Properti `initialRoute` menentukan *route* yang ditampilkan pertama kali ketika aplikasi dijalankan.
- Properti *routes* berisi peta (Map) antara nama *route* dan fungsi pembangun halaman.
- Properti `onGenerateRoute` digunakan ketika *route* perlu dibentuk secara dinamis, misalnya ketika nama *route* memuat parameter atau ketika argumen harus diteruskan ke konstruktor halaman.
- Properti `onUnknownRoute` menangani permintaan *route* yang tidak terdaftar sehingga aplikasi tidak berhenti secara paksa.

Penulisan nama *route* sebaiknya disimpan sebagai konstanta agar tidak terjadi kesalahan penulisan. Kesalahan satu karakter pada nama *route* hanya akan terdeteksi ketika aplikasi dijalankan, bukan ketika kode dikompilasi.

d. Widget Navigasi pada Material Design 3

Selain mekanisme perpindahan halaman, Flutter menyediakan sejumlah *widget* yang berfungsi sebagai kerangka navigasi tetap pada antarmuka. *Widget* ini tidak menambah tumpukan *route*, melainkan mengganti bagian isi halaman sesuai tujuan yang dipilih pengguna.

- `NavigationBar` merupakan bilah navigasi bawah pada Material Design 3 yang menggantikan `BottomNavigationBar`. *Widget* ini disarankan memuat tiga sampai lima tujuan utama, dengan properti `selectedIndex` sebagai penanda tujuan aktif dan `onDestinationSelected` sebagai penanganan pemilihan tujuan.
- `BottomAppBar` adalah bilah pada bagian bawah layar yang berperan sebagai wadah aksi, bukan sebagai penanda tujuan. *Widget* ini sering dipadukan dengan `FloatingActionButton` melalui properti `floatingActionButtonLocation` pada `Scaffold` serta dapat diberi bentuk cekungan menggunakan `CircularNotchedRectangle`.
- *Drawer* adalah panel navigasi yang muncul dari sisi layar dan cocok digunakan ketika jumlah tujuan cukup banyak atau ketika terdapat tujuan sekunder seperti pengaturan dan bantuan. `NavigationDrawer` merupakan versi Material Design 3 yang telah menyediakan penanda tujuan terpilih.
- `NavigationRail` adalah bilah navigasi vertikal pada sisi kiri layar yang dirancang untuk layar lebar seperti tablet, web, dan desktop. Properti `extended` dapat digunakan untuk menampilkan label secara penuh ketika ruang mencukupi.

Pemilihan *widget* navigasi bukan semata persoalan selera, melainkan berkaitan dengan lebar layar. Pedoman Material Design menyarankan penggunaan `NavigationBar` pada layar sempit dengan lebar di bawah 600 piksel logis, `NavigationRail` pada layar sedang, serta `NavigationDrawer` yang tampil menetap pada layar lebar di atas 840 piksel logis.

e. Tab Navigation

Tab navigation digunakan untuk mengelompokkan konten yang setara pada satu halaman yang sama, misalnya daftar tugas yang dipisahkan menurut status. Flutter menyediakan tiga

widget yang bekerja bersama, yaitu *TabBar* sebagai penunjuk *tab*, *TabBarView* sebagai penampil isi *tab*, dan *TabController* sebagai penyelaras keduanya. *DefaultTabController* dapat digunakan agar *TabController* dibuat secara otomatis tanpa penulisan kode tambahan.

Perlu dibedakan bahwa *tab navigation* memindahkan isi di dalam satu halaman, sedangkan *Navigator.push()* memindahkan pengguna ke halaman baru. Keduanya dapat digunakan bersamaan, misalnya bilah navigasi bawah untuk memilih bagian utama aplikasi dan *tab* untuk menyaring isi pada bagian tersebut.

f. Navigasi Adaptif

Aplikasi Flutter dapat berjalan pada HP, tablet, web, maupun desktop. Oleh karena itu, kerangka navigasi sebaiknya menyesuaikan diri terhadap ruang yang tersedia. *LayoutBuilder* atau *MediaQuery* dapat digunakan untuk membaca lebar layar, kemudian menentukan apakah *NavigationBar* atau *NavigationRail* yang ditampilkan. Penerapan pola ini memungkinkan satu berkas kode melayani beragam ukuran perangkat tanpa penggandaan halaman.

D. Alat dan Bahan

a. Perangkat Keras (Hardware)

- Komputer atau laptop dengan prosesor 64 bit, RAM minimal 8 GB (disarankan 16 GB apabila emulator dijalankan bersamaan dengan Android Studio), serta ruang penyimpanan bebas minimal 20 GB.
- Perangkat Android fisik beserta kabel data sebagai alternatif emulator, dengan *Developer Options* dan *USB Debugging* yang telah diaktifkan.
- Papan ketik dan tetikus untuk mempermudah pengujian navigasi pada target web dan desktop.

b. Perangkat Lunak (Software)

- Flutter SDK versi stabil beserta Dart SDK yang telah terpasang dan terverifikasi melalui perintah flutter doctor.
- Android Studio beserta Android SDK dan Android Virtual Device, atau Visual Studio Code dengan ekstensi Flutter dan Dart.
- Peramban Google Chrome untuk menguji tata letak navigasi pada layar lebar.
- Git sebagai perangkat kendali versi dan akun repositori untuk pengumpulan tugas.
- Sambungan internet untuk mengunduh paket melalui perintah flutter pub get.

E. Materi Praktikum

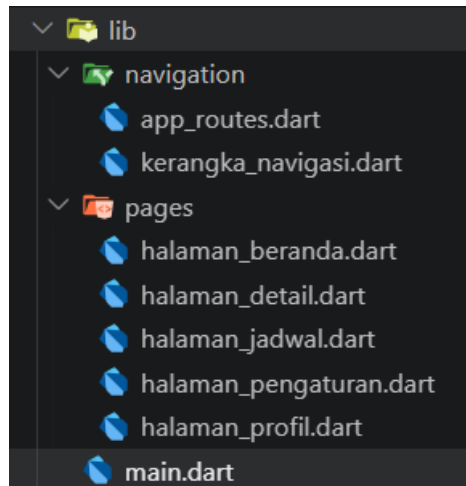
Materi praktikum pada modul ini disusun secara berjenjang. Bagian awal membahas mekanisme perpindahan halaman, kemudian dilanjutkan dengan pengaturan *route* secara terpusat, dan diakhiri dengan penyusunan kerangka navigasi tetap berupa bilah navigasi bawah, *drawer*, serta *navigation rail*. Setiap potongan kode dituliskan dalam berkas terpisah agar mudah diperiksa dan digunakan kembali.

a. Menyiapkan Proyek Praktikum

- Buka terminal, lalu buat proyek baru dengan perintah berikut. Ganti nim dengan nomor induk mahasiswa masing-masing.

```
flutter create navigasi_flutter_nim
cd navigasi_flutter_nim
flutter pub get
flutter run
```

- Pastikan aplikasi bawaan Flutter dapat berjalan terlebih dahulu sebelum kode diubah. Apabila aplikasi sudah tampil, hentikan proses dengan menekan tombol q pada terminal.
- Buat folder baru bernama pages dan navigation di dalam folder lib, sehingga struktur berkas menjadi seperti berikut.



Gambar 4.1 Struktur folder dalam lib

- Struktur berkas ini memisahkan halaman (pages) dari pengaturan navigasi (navigation). Pemisahan tersebut memudahkan penambahan halaman baru tanpa mengubah berkas main.dart.

b. Konsep Navigator dan Routes

Sebelum menuliskan kode, penting dipahami bahwa setiap halaman pada Flutter hanyalah sebuah *widget*. *Widget* tersebut menjadi sebuah *route* ketika dibungkus oleh objek *route*, misalnya *MaterialPageRoute*, kemudian diserahkan kepada Navigator. Navigator menyimpan *route* tersebut pada sebuah tumpukan dan menampilkan *route* paling atas kepada pengguna.

- Buka berkas lib/pages/halaman_beranda.dart, kemudian tuliskan kerangka halaman pertama berikut. Halaman ini masih belum memuat navigasi agar tampilan dapat diuji terlebih dahulu. Kode berikut adalah versi pertama dari berkas tersebut, dan berkas yang sama akan diperbarui pada subbagian selanjutnya.

```
import 'package:flutter/material.dart';

class HalamanBeranda extends StatelessWidget {
  const HalamanBeranda({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Beranda')),
      body: Center(
```

```

child: Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: const [
    Icon(Icons.home, size: 64, color: Colors.indigo),
    SizedBox(height: 12),
    Text('Halaman Beranda', style: TextStyle(fontSize: 20)),
  ],
),
),
);
}
}

```

- Perhatikan bahwa Scaffold digunakan sebagai kerangka halaman. Setiap *route* yang akan ditampilkan sebagai halaman penuh sebaiknya memiliki Scaffold sendiri agar AppBar dan tombol kembali muncul secara otomatis.
- Membuat berkas halaman saja belum mengubah tampilan aplikasi. Selama berkas `main.dart` masih berisi kode bawaan, yang dijalankan Flutter tetap aplikasi contoh berjudul Flutter Demo Home Page. Oleh sebab itu, buka berkas `lib/main.dart`, hapus seluruh isinya, kemudian ganti dengan kode berikut.

```

import 'package:flutter/material.dart';
import 'pages/halaman_beranda.dart';

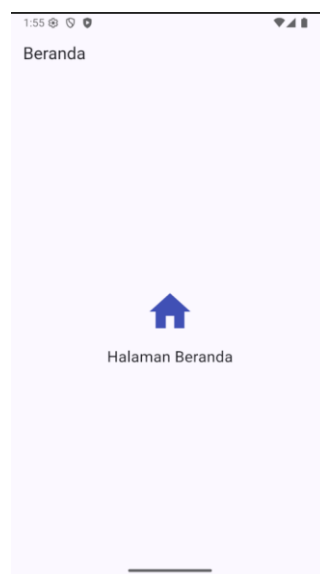
void main() {
  runApp(const AplikasiNavigasi());
}

class AplikasiNavigasi extends StatelessWidget {
  const AplikasiNavigasi({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum Navigasi Flutter',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        useMaterial3: true,
      ),
      home: const HalamanBeranda(),
    );
  }
}

```

- Fungsi main adalah titik awal aplikasi Dart, dan runApp memasang *widget* pertama ke layar. *Widget* pertama inilah yang menggantikan aplikasi contoh bawaan Flutter.
- MaterialApp adalah pembungkus aplikasi yang secara otomatis menyisipkan Navigator pada akar aplikasi, sehingga navigasi antar halaman dapat digunakan pada subbagian berikutnya.
- Properti home menentukan halaman yang tampil pertama kali, dalam hal ini HalamanBeranda. Properti debugShowCheckedModeBanner bernilai false untuk menghilangkan tulisan *DEBUG* pada sudut kanan atas.
- Baris import 'pages/halaman_beranda.dart' menghubungkan main.dart dengan berkas halaman. Alamat berkas ditulis relatif terhadap posisi main.dart, sehingga kata lib tidak perlu dituliskan.
- Setelah berkas disimpan, lakukan *hot restart* dengan menekan tombol R kapital pada terminal, bukan hanya *hot reload*. Perubahan pada fungsi main memerlukan *hot restart* agar aplikasi dimuat ulang dari awal.



Gambar 4.2 Tampilan halaman beranda sebelum navigasi ditambahkan

c. Implementasi Navigasi Sederhana dengan Navigator.push() dan Navigator.pop()

- Buat berkas lib/pages/halaman_detail.dart sebagai halaman tujuan. Halaman ini menerima data melalui konstruktor dan dapat mengembalikan nilai ketika ditutup.

```
import 'package:flutter/material.dart';

class HalamanDetail extends StatelessWidget {
  const HalamanDetail({super.key, required this.judul, required this.keterangan});

  final String judul;
  final String keterangan;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(judul)),
```

```

body: Padding(
  padding: const EdgeInsets.all(16),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(judul, style: const TextStyle(fontSize: 22, fontWeight: FontWeight.bold)),
      const SizedBox(height: 8),
      Text(keterangan, style: const TextStyle(fontSize: 14)),
      const Spacer(),
      Row(
        children: [
          Expanded(
            child: OutlinedButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Kembali'),
            ),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: ElevatedButton(
              onPressed: () => Navigator.pop(context, 'Data dari halaman detail'),
              child: const Text('Kembali dengan Nilai'),
            ),
          ),
        ],
      ),
    ],
  ),
);
}
}

```

- Terdapat dua bentuk pemanggilan `Navigator.pop()` pada kode di atas. Bentuk pertama hanya menutup halaman, sedangkan bentuk kedua menutup halaman sekaligus mengirim nilai kembali ke halaman pemanggil.
- Ganti seluruh isi berkas `lib/pages/halaman_beranda.dart` dengan kode berikut, sehingga versi ini menggantikan kode pada subbagian sebelumnya. Berkas diubah menjadi `StatefulWidget` agar halaman dapat menampilkan nilai yang dikirim balik oleh halaman detail. Perhatikan penggunaan kata kunci `await` pada `Navigator.push()`.

```

import 'package:flutter/material.dart';
import 'halaman_detail.dart';

class HalamanBeranda extends StatefulWidget {
  const HalamanBeranda({super.key});

  @override
  State<HalamanBeranda> createState() => _HalamanBerandaState();
}

class _HalamanBerandaState extends State<HalamanBeranda> {
  String _pesan = 'Belum ada data yang dikirim balik.';

  Future<void> _bukaDetail() async {
    final hasil = await Navigator.push<String>(
      context,
      MaterialPageRoute(
        builder: (context) => const HalamanDetail(
          judul: 'Detail Mata Kuliah',
          keterangan: 'Pemrograman Perangkat Bergerak Lanjut, 4 SKS, Semester 5.',
        ),
      ),
    );

    if (!mounted) return;
    setState(() {
      _pesan = hasil ?? 'Halaman detail ditutup tanpa mengirim nilai.';
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Beranda')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(Icons.home, size: 64, color: Colors.indigo),
            const SizedBox(height: 12),
            Text(_pesan, textAlign: TextAlign.center),
          ],
        ),
      ),
    );
  }
}

```

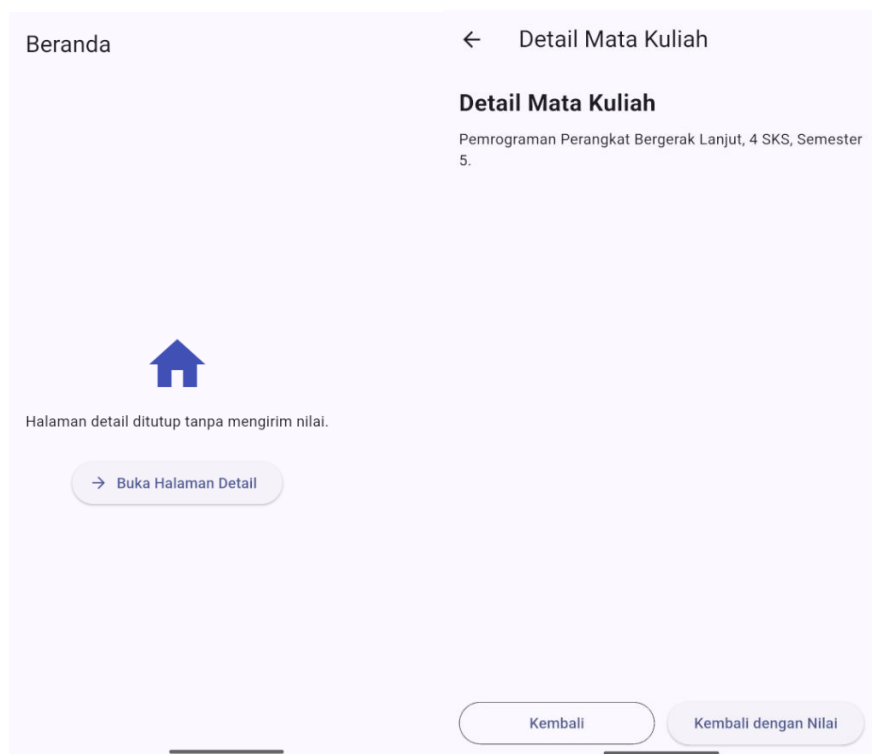
```

const SizedBox(height: 24),
ElevatedButton.icon(
  onPressed: _bukaDetail,
  icon: const Icon(Icons.arrow_forward),
  label: const Text('Buka Halaman Detail'),
),
],
),
),
);
}
}

```

- `Navigator.push()` menghasilkan objek `Future`, sehingga nilai kembalian hanya dapat diambil setelah halaman tujuan ditutup. Pemeriksaan `mounted` diperlukan untuk memastikan *widget* masih berada pada pohon *widget* sebelum `setState()` dipanggil.
- Jalankan aplikasi, tekan tombol Buka Halaman Detail, kemudian coba kedua tombol pada halaman detail. Amati perbedaan teks yang muncul pada halaman beranda.

Perlu diperhatikan bahwa tombol kembali pada `AppBar` halaman detail muncul secara otomatis. Tombol tersebut dibuat oleh Flutter karena `Navigator` mendeteksi masih terdapat *route* di bawahnya. Apabila halaman detail tidak menampilkan tombol kembali, hal tersebut menandakan halaman dibuka menggunakan `pushReplacement()` dan bukan `push()`.



Gambar 4.3 Tampilan halaman beranda sesudah navigasi dan pengiriman data balik ditambahkan

d. Navigasi dengan Named Routes

Named routes memusatkan pendaftaran halaman pada satu berkas sehingga pemanggilan navigasi menjadi lebih ringkas. Pada langkah berikut, seluruh nama *route* disimpan sebagai konstanta agar terhindar dari kesalahan penulisan.

- Buat berkas `lib/navigation/app_routes.dart`, kemudian tuliskan kode berikut.

```
// lib/navigation/app_routes.dart
import 'package:flutter/material.dart';
import '../pages/halaman_beranda.dart';
import '../pages/halaman_detail.dart';

class AppRoutes {
  static const String beranda = '/';
  static const String detail = '/detail';

  // Route tanpa argumen didaftarkan di sini.
  static Map<String, WidgetBuilder> daftarRoute() {
    return {
      beranda: (context) => const HalamanBeranda(),
    };
  }

  // Route yang memerlukan argumen dibentuk di sini.
  static Route<dynamic>? bentukRoute(RouteSettings settings) {
    if (settings.name == detail) {
      final argumen = settings.arguments as Map<String, String>? ?? const {};
      return MaterialPageRoute(
        builder: (context) => HalamanDetail(
          judul: argumen['judul'] ?? 'Tanpa Judul',
          keterangan: argumen['keterangan'] ?? 'Tidak ada keterangan.',
        ),
      );
    }
    return null;
  }

  // Dipakai ketika nama route tidak terdaftar.
  static Route<dynamic> routeTidakDikenal(RouteSettings settings) {
    return MaterialPageRoute(
      builder: (context) => Scaffold(
        appBar: AppBar(title: const Text('Route Tidak Ditemukan')),
        body: Center(child: Text('Route ${settings.name} belum terdaftar.')),
      ),
    );
  }
}
```

```

    ),
  );
}
}

```

- Perhatikan bahwa berkas ini hanya mendaftarkan halaman yang sudah dibuat sampai tahap ini, yaitu halaman beranda dan halaman detail. Mendaftarkan halaman yang berkasnya belum ada akan menimbulkan galat Target of URI does not exist, sehingga aplikasi tidak dapat dijalankan.
- Halaman lain seperti jadwal, profil, dan pengaturan akan didaftarkan menyusul pada subbagian berikutnya, setelah berkas halamannya dibuat. Begitu pula *route* beranda yang nantinya diarahkan ke kerangka navigasi, bukan langsung ke halaman beranda.
- Agar nama *route* dikenali di halaman yang memanggilnya, tambahkan baris **import** `'../navigation/app_routes.dart'` pada bagian atas berkas `lib/pages/halaman_beranda.dart`.
- Metode `daftarRoute()` digunakan untuk halaman tanpa argumen, sedangkan `bentukRoute()` digunakan untuk halaman yang memerlukan argumen. Pemisahan ini merupakan praktik yang lazim karena properti *routes* tidak dapat menerima argumen secara langsung.
- Hubungkan berkas tersebut pada `MaterialApp` di dalam `lib/main.dart` seperti berikut.

```

import 'package:flutter/material.dart';
import 'navigation/app_routes.dart';

void main() => runApp(const AplikasiNavigasi());

class AplikasiNavigasi extends StatelessWidget {
  const AplikasiNavigasi({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum Navigasi Flutter',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        useMaterial3: true,
      ),
      initialRoute: AppRoutes.beranda,
      routes: AppRoutes.daftarRoute(),
      onGenerateRoute: AppRoutes.bentukRoute,
      onUnknownRoute: AppRoutes.routeTidakDikenal,
    );
  }
}

```

- Sampai tahap ini halaman beranda masih berpindah halaman dengan cara lama, yaitu `Navigator.push` disertai `MaterialPageRoute` dan penyebutan nama kelas halaman tujuan secara langsung. Agar seluruh perpindahan halaman melewati satu pintu, ganti seluruh isi `lib/pages/halaman_beranda.dart` dengan kode berikut. Ini adalah versi ketiga berkas tersebut.

```
// lib/pages/halaman_beranda.dart
import 'package:flutter/material.dart';
import '../navigation/app_routes.dart';

class HalamanBeranda extends StatefulWidget {
  const HalamanBeranda({super.key});

  @override
  State<HalamanBeranda> createState() => _HalamanBerandaState();
}

class _HalamanBerandaState extends State<HalamanBeranda> {
  String _pesan = 'Belum ada data yang dikirim balik.';

  Future<void> _bukaDetail() async {
    // Perpindahan halaman kini menyebut NAMA route, bukan kelas halaman.
    final hasil = await Navigator.pushNamed(
      context,
      AppRoutes.detail,
      arguments: {
        'judul': 'Detail Mata Kuliah',
        'keterangan': 'Pertemuan ke-2 membahas navigasi dan routing.',
      },
    );

    if (!mounted) return;
    setState() {
      _pesan = hasil is String ? hasil : 'Halaman ditutup tanpa mengirim data.';
    });
  }

  void _ujiRouteSalah() {
    Navigator.pushNamed(context, '/salah');
  }

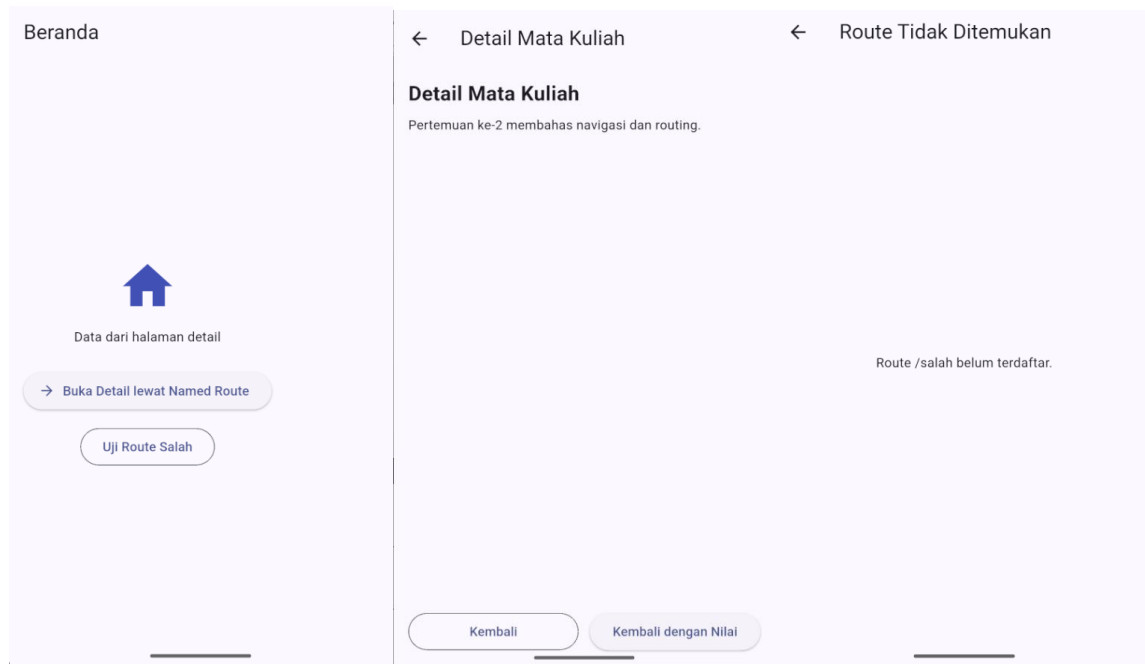
  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```

appBar: AppBar(title: const Text('Beranda')),
body: Padding(
  padding: const EdgeInsets.all(16),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      const Icon(Icons.home, size: 64, color: Colors.indigo),
      const SizedBox(height: 12),
      Text(_pesan, textAlign: TextAlign.center),
      const SizedBox(height: 24),
      ElevatedButton.icon(
        onPressed: _bukaDetail,
        icon: const Icon(Icons.arrow_forward),
        label: const Text('Buka Detail lewat Named Route'),
      ),
      const SizedBox(height: 12),
      OutlinedButton(
        onPressed: _ujiRouteSalah,
        child: const Text('Uji Route Salah'),
      ),
    ],
  ),
),
);
}
}

```

- Baris `import '../navigation/app_routes.dart'` menggantikan `import halaman_detail.dart`. Halaman beranda kini tidak perlu mengetahui nama kelas halaman tujuan, cukup mengetahui nama *route*-nya.
- Nilai kembalian `Navigator.pushNamed` bertipe `Object?`, bukan `String`, karena Flutter tidak dapat memastikan jenis data yang dikirim balik oleh *route* bernama. Oleh sebab itu dipakai pemeriksaan hasil `is String` sebelum nilai tersebut ditampilkan.
- Tombol *Uji Route Salah* sengaja memanggil nama *route* yang tidak terdaftar agar mahasiswa dapat melihat cara kerja `onUnknownRoute` secara langsung.
- Halaman detail tidak perlu diubah. Argumen sudah disesuaikan dalam metode `bentukRoute` pada `app_routes.dart`, kemudian diteruskan ke konstruktor `HalamanDetail`.



Gambar 4.4 Hasil pengujian named routes beserta penanganan route yang tidak terdaftar

Tabel 4.1 Perbandingan `Navigator.push` dan `Navigator.pushNamed`

Aspek	<code>Navigator.push</code>	<code>Navigator.pushNamed</code>
Cara menyebut tujuan	Menyebut kelas halaman langsung di dalam <code>MaterialPageRoute</code>	Menyebut nama route berupa teks yang sudah didaftarkan
Kebutuhan import	Setiap halaman pemanggil mengimpor berkas halaman tujuan	Hanya <code>app_routes.dart</code> yang mengimpor berkas halaman
Cara mengirim data	Melalui konstruktor halaman tujuan	Melalui properti <code>arguments</code> , dibongkar di <code>onGenerateRoute</code> atau di halaman tujuan
Keamanan tipe data	Diperiksa saat kode dikompilasi	Bertipe <code>Object?</code> sehingga perlu diperiksa saat aplikasi berjalan
Kesalahan penulisan	Nama kelas salah langsung ditandai editor	Nama route salah baru terlihat saat dijalankan, sehingga perlu konstanta dan <code>onUnknownRoute</code>
Nilai kembalian	<code>Navigator.pop(context, nilai)</code> ditunggu dengan <code>await</code>	Sama, <code>Navigator.pop(context, nilai)</code> ditunggu dengan <code>await</code>
Paling cocok dipakai	Halaman sekali pakai atau dialog yang dibuka dari satu tempat	Aplikasi dengan banyak halaman, menu, drawer, dan tautan langsung

e. Membuat Bottom Navigation Bar dengan NavigationBar

Bilah navigasi bawah tidak menambah *route* pada tumpukan Navigator. *Widget* ini hanya mengganti bagian isi Scaffold sesuai indeks tujuan yang dipilih. Oleh karena itu, kerangka navigasi harus berupa *StatefulWidget* agar indeks terpilih dapat disimpan.

- Kerangka navigasi memerlukan tiga berkas halaman yang belum dibuat, yaitu halaman jadwal, halaman profil, dan halaman pengaturan. Apabila ketiganya belum ada, akan muncul galat Target of URI does not exist begitu kerangka navigasi dijalankan. Oleh sebab itu buat ketiga berkas berikut terlebih dahulu dalam bentuk sederhana. Isinya akan dilengkapi pada subbagian Membuat *Tab Navigation* dan Melengkapi Halaman Pendukung.

```
// lib/pages/halaman_jadwal.dart (versi sederhana)
import 'package:flutter/material.dart';

class HalamanJadwal extends StatelessWidget {
  const HalamanJadwal({super.key});

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text('Halaman Jadwal', style: TextStyle(fontSize: 20)),
    );
  }
}
```

```
// lib/pages/halaman_profil.dart (versi sederhana)
import 'package:flutter/material.dart';

class HalamanProfil extends StatelessWidget {
  const HalamanProfil({super.key});

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text('Halaman Profil', style: TextStyle(fontSize: 20)),
    );
  }
}
```

```
// lib/pages/halaman_pengaturan.dart (versi sederhana)
import 'package:flutter/material.dart';

class HalamanPengaturan extends StatelessWidget {
```

```
const HalamanPengaturan({super.key});

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Pengaturan')),
    body: const Center(child: Text('Halaman Pengaturan')),
  );
}
}
```

- Perhatikan perbedaan penting pada ketiga berkas di atas. Halaman jadwal dan halaman profil tidak memakai Scaffold, sebab keduanya akan ditampilkan di dalam Scaffold milik kerangka navigasi. Halaman pengaturan memakai Scaffold karena dibuka sebagai halaman penuh melalui *named route*, sehingga memerlukan AppBar dan tombol kembali sendiri.
- Apabila halaman jadwal atau halaman profil diberi Scaffold, akan tampak dua AppBar bertumpuk pada satu layar. Ini kekeliruan yang sering terjadi dan tidak menimbulkan galat, tetapi tampilannya salah.
- Sesudah berkas `kerangka_navigasi.dart` selesai dibuat, perbarui berkas `lib/navigation/app_routes.dart` agar *route* beranda mengarah ke kerangka navigasi, bukan langsung ke halaman beranda.

```
// lib/navigation/app_routes.dart (versi lengkap, tulis ulang seluruh berkas)
import 'package:flutter/material.dart';
import '../pages/halaman_detail.dart';
import '../pages/halaman_pengaturan.dart';
import 'kerangka_navigasi.dart';

class AppRoutes {
  static const String beranda = '/';
  static const String pengaturan = '/pengaturan';
  static const String detail = '/detail';

  static Map<String, WidgetBuilder> daftarRoute() {
    return {
      // route beranda kini mengarah ke kerangka navigasi
      beranda: (context) => const KerangkaNavigasi(),
      pengaturan: (context) => const HalamanPengaturan(),
    };
  }

  static Route<dynamic>? bentukRoute(RouteSettings settings) {
    if (settings.name == detail) {
```

```

final argumen = settings.arguments as Map<String, String>? ?? const {};
return MaterialPageRoute(
  builder: (context) => HalamanDetail(
    judul: argumen['judul'] ?? 'Tanpa Judul',
    keterangan: argumen['keterangan'] ?? 'Tidak ada keterangan.',
  ),
);
}
return null;
}

static Route<dynamic> routeTidakDikenal(RouteSettings settings) {
  return MaterialPageRoute(
    builder: (context) => Scaffold(
      appBar: AppBar(title: const Text('Route Tidak Ditemukan')),
      body: Center(child: Text('Route ${settings.name} belum terdaftar.')),
    ),
  );
}
}

```

- Baris import halaman_beranda.dart pada app_routes.dart sudah tidak diperlukan lagi, sebab halaman beranda kini dipanggil dari dalam kerangka navigasi. Baris import halaman_pengaturan.dart baru ditambahkan setelah berkas halaman pengaturan dibuat pada subbagian Melengkapi Halaman Pendukung.
- Buat berkas lib/navigation/kerangka_navigasi.dart, kemudian tuliskan kode berikut.

```

import 'package:flutter/material.dart';
import '../pages/halaman_beranda.dart';
import '../pages/halaman_jadwal.dart';
import '../pages/halaman_profil.dart';

class KerangkaNavigasi extends StatefulWidget {
  const KerangkaNavigasi({super.key});

  @override
  State<KerangkaNavigasi> createState() => _KerangkaNavigasiState();
}

class _KerangkaNavigasiState extends State<KerangkaNavigasi> {
  int _indeksTerpilih = 0;

  final List<Widget> _halaman = const [

```

```

    HalamanBeranda(),
    HalamanJadwal(),
    HalamanProfil(),
];

final List<String> _judul = const ['Beranda', 'Jadwal', 'Profil'];

void _pilihTujuan(int indeks) {
    setState(() => _indeksTerpilih = indeks);
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: Text(_judul[_indeksTerpilih])),
        body: _halaman[_indeksTerpilih],
        bottomNavigationBar: NavigationBar(
            selectedIndex: _indeksTerpilih,
            onDestinationSelected: _pilihTujuan,
            destinations: const [
                NavigationDestination(
                    icon: Icon(Icons.home_outlined),
                    selectedIcon: Icon(Icons.home),
                    label: 'Beranda',
                ),
                NavigationDestination(
                    icon: Icon(Icons.calendar_month_outlined),
                    selectedIcon: Icon(Icons.calendar_month),
                    label: 'Jadwal',
                ),
                NavigationDestination(
                    icon: Icon(Icons.person_outline),
                    selectedIcon: Icon(Icons.person),
                    label: 'Profil',
                ),
            ],
        ),
    );
}

```

- Properti `selectedIndex` menentukan tujuan yang sedang aktif, sedangkan `onDestinationSelected` menerima indeks tujuan yang ditekan pengguna. Jumlah item pada `destinations` harus sama dengan jumlah halaman pada daftar `_halaman`, sebab ketidaksesuaian jumlah menyebabkan galat `RangeError` ketika tujuan terakhir dipilih.
- *Widget* `NavigationDestination` menyediakan properti `selectedIcon` sehingga ikon dapat berubah dari bentuk garis luar menjadi bentuk terisi ketika tujuan sedang aktif. Perilaku ini merupakan ciri khas Material Design 3.
- Langkah terakhir pada subbagian ini adalah memeriksa berkas `lib/main.dart`. Berkas inilah yang menentukan halaman mana yang dijalankan pertama kali. Apabila isinya masih kode bawaan Flutter, bilah navigasi bawah tidak akan pernah muncul walaupun kerangka navigasi sudah dibuat. Pastikan isi berkas `main.dart` tepat seperti berikut.

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'navigation/app_routes.dart';

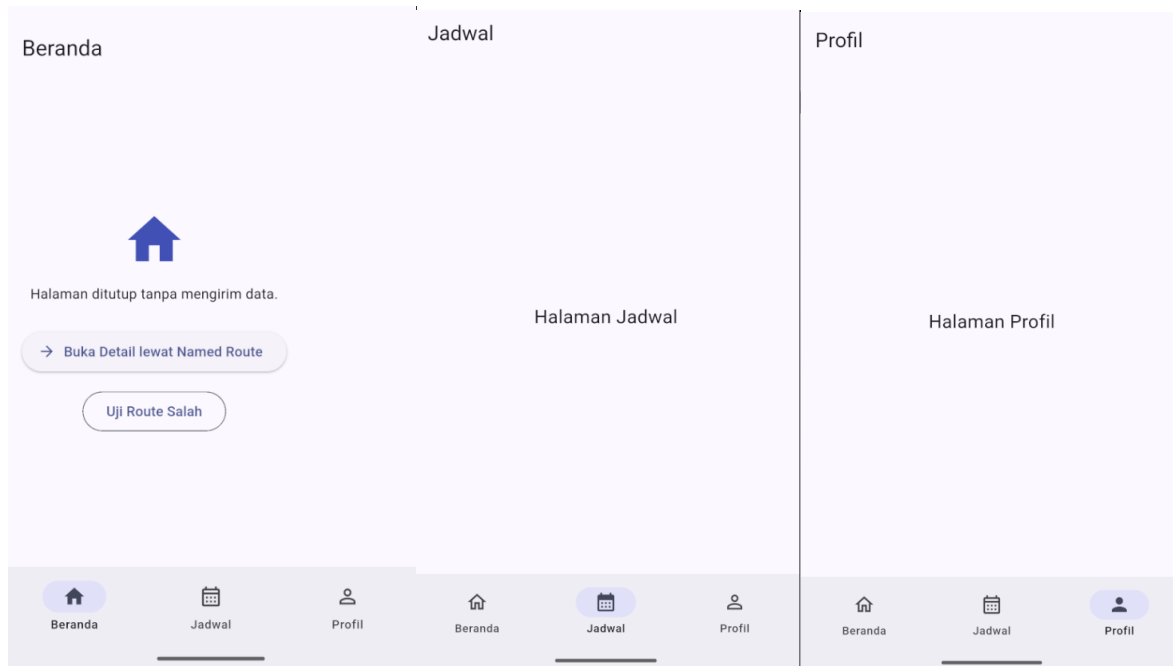
void main() {
  runApp(const AplikasiNavigasi());
}

class AplikasiNavigasi extends StatelessWidget {
  const AplikasiNavigasi({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum Navigasi Flutter',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        useMaterial3: true,
      ),

      // '/' terdaftar pada AppRoutes dan kini mengarah ke KerangkaNavigasi,
      // bukan lagi ke HalamanBeranda.
      initialRoute: AppRoutes.beranda,
      routes: AppRoutes.daftarRoute(),
      onGenerateRoute: AppRoutes.bentukRoute,
      onUnknownRoute: AppRoutes.routeTidakDikenal,
    );
  }
}
```

- Alur pemanggilannya adalah `main.dart` membuka `initialRoute` bernilai `'/'`, lalu nama *route* tersebut dicari pada `AppRoutes.daftarRoute()`, dan hasilnya adalah `KerangkaNavigasi`. `Kerangka navigasi` inilah yang menampilkan `Scaffold` beserta bilah navigasi bawah.



Gambar 4.5 Anatomi `NavigationBar`: `AppBar`, area isi, dan tiga tujuan navigasi dengan tujuan pertama aktif

f. Menggunakan `BottomAppBar` sebagai Wadah Aksi

`BottomAppBar` berbeda peran dengan `NavigationBar`. *Widget* ini tidak dipakai untuk memindahkan tujuan utama, melainkan untuk menempatkan aksi yang berkaitan dengan halaman yang sedang tampil, misalnya menyaring data, membagikan, atau menambah catatan. `BottomAppBar` sering dipadukan dengan `FloatingActionButton` yang berada pada cekungan bilah.

- Berkas `kerangka_navigasi.dart` JANGAN diubah pada subbagian ini, sebab bilah navigasi bawah yang sudah dibuat akan hilang. `BottomAppBar` dicoba pada halaman terpisah. Buat berkas baru bernama `lib/pages/halaman_catatan.dart`, kemudian tuliskan kode berikut secara lengkap.

```
// lib/pages/halaman_catatan.dart (berkas BARU)
import 'package:flutter/material.dart';

class HalamanCatatan extends StatelessWidget {
  const HalamanCatatan({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan')),
      body: const Center(child: Text('Daftar catatan kuliah')),
    );
  }
}
```

```
// tombol melayang yang menempel pada cekungan bilah
floatingActionButton: FloatingActionButton(
  onPressed: () {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Tambah catatan baru')),
    );
  },
  child: const Icon(Icons.add),
),
floatingActionButtonLocation:
FloatingActionButtonLocation.centerDocked,

// BottomAppBar berisi aksi, bukan tujuan navigasi
bottomNavigationBar: BottomAppBar(
  shape: const CircularNotchedRectangle(),
  notchMargin: 8,
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceAround,
    children: [
      IconButton(icon: const Icon(Icons.search), onPressed: () {}),
      IconButton(icon: const Icon(Icons.filter_list), onPressed: () {}),
      const SizedBox(width: 40),
      IconButton(icon: const Icon(Icons.sort), onPressed: () {}),
      IconButton(icon: const Icon(Icons.more_vert), onPressed: () {}),
    ],
  ),
),
);
}
}
```

- Properti `shape` bernilai `CircularNotchedRectangle()` menghasilkan cekungan pada bilah, sedangkan `notchMargin` mengatur jarak antara cekungan dengan tombol. `SizedBox` dengan lebar tertentu disisipkan pada tengah baris agar ikon tidak tertutup oleh `FloatingActionButton`.
- Halaman catatan belum dapat dibuka karena belum terdaftar sebagai *route*. Buka berkas `lib/navigation/app_routes.dart`, kemudian lakukan tiga penambahan berikut. Bagian lain pada berkas tersebut tidak diubah.

```
// lib/navigation/app_routes.dart

// TAMBAHAN 1 - pada bagian import paling atas
import '../pages/halaman_catatan.dart';
```

```
// TAMBAHAN 2 - pada kelompok konstanta nama route
static const String catatan = '/catatan';

// TAMBAHAN 3 - di dalam metode daftarRoute, sesudah baris pengaturan
catatan: (context) => const HalamanCatatan(),
```

- Terakhir, buat jalan masuk ke halaman catatan. Buka berkas lib/pages/halaman_profil.dart, kemudian tulis ulang seluruh isinya dengan kode berikut. Berkas ini dituliskan utuh karena ada dua hal yang berubah sekaligus, yaitu penambahan baris import dan penambahan tombol. Selain itu, kata kunci const pada daftar children versi sederhana harus dihapus, sebab tombol yang memiliki onPressed tidak dapat diletakkan di dalam daftar bertanda const.

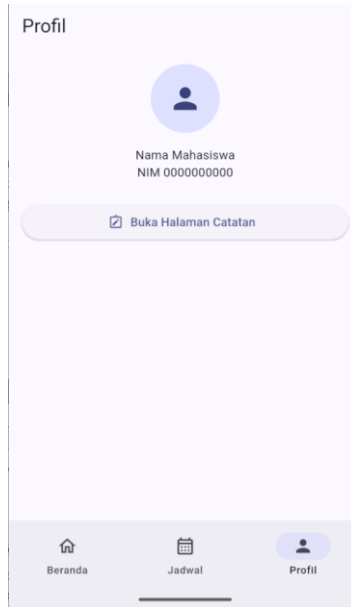
```
// lib/pages/halaman_profil.dart (tulis ulang seluruh berkas)
import 'package:flutter/material.dart';
import '../navigation/app_routes.dart';

class HalamanProfil extends StatelessWidget {
  const HalamanProfil({super.key});

  @override
  Widget build(BuildContext context) {
    // Tanpa Scaffold, sebab halaman ini tampil di dalam kerangka navigasi.
    return ListView(
      padding: const EdgeInsets.all(16),
      children: [
        const CircleAvatar(radius: 40, child: Icon(Icons.person, size: 40)),
        const SizedBox(height: 16),
        const Center(child: Text('Nama Mahasiswa')),
        const Center(child: Text('NIM 0000000000')),
        const SizedBox(height: 24),

        // Tombol inilah jalan masuk ke halaman catatan.
        ElevatedButton.icon(
          onPressed: () => Navigator.pushNamed(context, AppRoutes.catatan),
          icon: const Icon(Icons.note_alt_outlined),
          label: const Text('Buka Halaman Catatan'),
        ),
      ],
    );
  }
}
```

- Perhatikan letak kata kunci `const` pada kode di atas. Kata kunci tersebut kini menempel pada masing-masing *widget* yang isinya tetap, misalnya `const CircleAvatar` dan `const SizedBox`, dan bukan pada daftar `children` secara keseluruhan. Inilah cara mencampur *widget* tetap dengan *widget* yang memiliki aksi di dalam satu daftar.
- Halaman profil tetap tidak memakai `Scaffold`, sebab halaman ini tampil di dalam `Scaffold` milik kerangka navigasi. Yang memakai `Scaffold` adalah halaman catatan, karena dibuka sebagai halaman penuh.



Gambar 4.6 `BottomAppBar` dengan cekungan `CircularNotchedRectangle` dan `FloatingActionButton` pada posisi `centerDocked`

g. Membuat Navigation Drawer

Drawer digunakan ketika jumlah tujuan melebihi kapasitas bilah navigasi bawah atau ketika terdapat tujuan sekunder. *Drawer* dipasang melalui properti `drawer` pada `Scaffold`, dan Flutter secara otomatis menambahkan ikon menu pada `AppBar` untuk membukanya.

- Buka berkas `lib/navigation/kerangka_navigasi.dart`, kemudian tulis ulang seluruh isinya dengan kode berikut. Dua bagian yang baru sudah ditandai komentar BARU di dalam kode, yaitu baris `drawer` pada `Scaffold` dan metode `_buatDrawer` di bagian bawah kelas. Urutan properti pada `Scaffold` adalah `appBar`, `drawer`, `body`, kemudian `bottomNavigationBar`.

```
// lib/navigation/kerangka_navigasi.dart (tulis ulang seluruh berkas)
import 'package:flutter/material.dart';
import '../pages/halaman_beranda.dart';
import '../pages/halaman_jadwal.dart';
import '../pages/halaman_profil.dart';
import 'app_routes.dart';

class KerangkaNavigasi extends StatefulWidget {
  const KerangkaNavigasi({super.key});

  @override
```

```

State<KerangkaNavigasi> createState() => _KerangkaNavigasiState();
}

class _KerangkaNavigasiState extends State<KerangkaNavigasi> {
  int _indeksTerpilih = 0;

  final List<Widget> _halaman = const [
    HalamanBeranda(),
    HalamanJadwal(),
    HalamanProfil(),
  ];

  final List<String> _judul = const ['Beranda', 'Jadwal', 'Profil'];

  void _pilihTujuan(int indeks) {
    setState(() {
      _indeksTerpilih = indeks;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(_judul[_indeksTerpilih])),

      // ===== BARU (1) : satu baris drawer, diletakkan sesudah appBar =====
      drawer: _buatDrawer(),

      body: _halaman[_indeksTerpilih],
      bottomNavigationBar: NavigationBar(
        selectedIndex: _indeksTerpilih,
        onDestinationSelected: _pilihTujuan,
        destinations: const [
          NavigationDestination(
            icon: Icon(Icons.home_outlined),
            selectedIcon: Icon(Icons.home),
            label: 'Beranda',
          ),
          NavigationDestination(
            icon: Icon(Icons.calendar_month_outlined),
            selectedIcon: Icon(Icons.calendar_month),
            label: 'Jadwal',

```

```

    ),
    NavigationDestination(
      icon: Icon(Icons.person_outline),
      selectedIcon: Icon(Icons.person),
      label: 'Profil',
    ),
  ],
),
);
}

// ===== BARU (2) : metode drawer, diletakkan di bawah metode build =====
Widget _buatDrawer() {
  return NavigationDrawer(
    selectedIndex: _indeksTerpilih,
    onDestinationSelected: (indeks) {
      _pilihTujuan(indeks);
      Navigator.pop(context); // menutup drawer
    },
    children: [
      const UserAccountsDrawerHeader(
        accountName: Text('Nama Mahasiswa'),
        accountEmail: Text('mahasiswa@kampus.ac.id'),
        currentAccountPicture: CircleAvatar(child: Icon(Icons.person)),
      ),
      const Padding(
        padding: EdgeInsets.fromLTRB(28, 16, 16, 10),
        child: Text('Menu Utama'),
      ),
      const NavigationDrawerDestination(
        icon: Icon(Icons.home_outlined),
        selectedIcon: Icon(Icons.home),
        label: Text('Beranda'),
      ),
      const NavigationDrawerDestination(
        icon: Icon(Icons.calendar_month_outlined),
        selectedIcon: Icon(Icons.calendar_month),
        label: Text('Jadwal'),
      ),
      const NavigationDrawerDestination(
        icon: Icon(Icons.person_outline),
        selectedIcon: Icon(Icons.person),

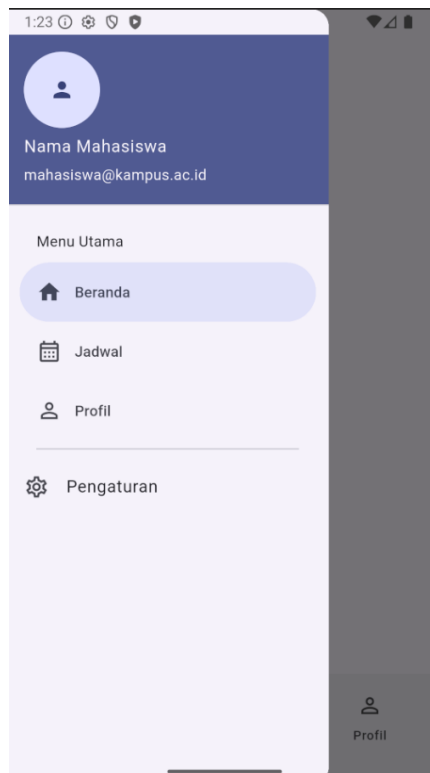
```

```

        label: Text('Profil'),
      ),
      const Divider(indent: 28, endIndent: 28),
      ListTile(
        leading: const Icon(Icons.settings_outlined),
        title: const Text('Pengaturan'),
        onTap: () {
          Navigator.pop(context); // tutup drawer dahulu
          Navigator.pushNamed(context, AppRoutes.pengaturan);
        },
      ),
    ],
  );
}
}

```

- Ringkasan perubahan pada subbagian ini hanya dua, yaitu satu baris *drawer*: `_buatDrawer()` di dalam Scaffold, dan satu metode baru bernama `_buatDrawer` yang diletakkan di dalam kelas `_KerangkaNavigasiState`, sesudah metode `build` ditutup. Berkas lain tidak ada yang diubah.
- Metode `_buatDrawer` harus berada di dalam kelas `_KerangkaNavigasiState`, bukan di luarnya. Cirinya, metode tersebut berada sebelum tanda kurung kurawal penutup terakhir pada berkas. Apabila diletakkan di luar kelas, akan muncul galat karena variabel `_indeksTerpilih` dan fungsi `_pilihTujuan` tidak dikenali.



Gambar 4.7 Struktur NavigationDrawer: header akun, kelompok tujuan utama, dan tujuan sekunder

h. Membuat Navigation Rail untuk Layar Lebar

NavigationRail menempatkan tujuan navigasi pada sisi kiri layar. *Widget* ini bukan properti Scaffold, melainkan *widget* biasa yang diletakkan di dalam Row bersama isi halaman. Pola yang lazim digunakan adalah NavigationRail, VerticalDivider, kemudian Expanded yang memuat isi halaman.

- Buka berkas lib/navigation/kerangka_navigasi.dart, kemudian tulis ulang seluruh isinya dengan kode berikut. Kode ini adalah versi final kerangka navigasi, yaitu sudah memuat NavigationBar, NavigationDrawer, dan NavigationRail sekaligus. Bagian yang baru pada subbagian ini ditandai komentar BARU di dalam kode, yaitu metode `_tataLetakLebar` dan perubahan pada metode `build`.

```
// lib/navigation/kerangka_navigasi.dart (VERSI FINAL, tulis ulang seluruh berkas)
import 'package:flutter/material.dart';
import '../pages/halaman_beranda.dart';
import '../pages/halaman_jadwal.dart';
import '../pages/halaman_profil.dart';
import 'app_routes.dart';

class KerangkaNavigasi extends StatefulWidget {
  const KerangkaNavigasi({super.key});

  @override
  State<KerangkaNavigasi> createState() => _KerangkaNavigasiState();
}

class _KerangkaNavigasiState extends State<KerangkaNavigasi> {
  int _indeksTerpilih = 0;

  final List<Widget> _halaman = const [
    HalamanBeranda(),
    HalamanJadwal(),
    HalamanProfil(),
  ];

  final List<String> _judul = const ['Beranda', 'Jadwal', 'Profil'];

  void _pilihTujuan(int indeks) {
    setState(() {
      _indeksTerpilih = indeks;
    });
  }

  @override
```

```

Widget build(BuildContext context) {
  // ===== BARU (1) : membaca lebar layar untuk menentukan tata letak =====
  final double lebar = MediaQuery.of(context).size.width;
  final bool layarLebar = lebar >= 600;

  return Scaffold(
    appBar: AppBar(title: Text(_judul[_indeksTerpilih])),
    drawer: _buatDrawer(),

    // ===== BARU (2) : isi halaman mengikuti lebar layar =====
    body: layarLebar ? _tataLetakLebar() : _halaman[_indeksTerpilih],

    // ===== BARU (3) : bilah bawah disembunyikan pada layar lebar =====
    bottomNavigationBar: layarLebar ? null : _buatBilahBawah(),
  );
}

// Bilah navigasi bawah, dipakai hanya pada layar sempit.
Widget _buatBilahBawah() {
  return NavigationBar(
    selectedIndex: _indeksTerpilih,
    onDestinationSelected: _pilihTujuan,
    destinations: const [
      NavigationDestination(
        icon: Icon(Icons.home_outlined),
        selectedIcon: Icon(Icons.home),
        label: 'Beranda',
      ),
      NavigationDestination(
        icon: Icon(Icons.calendar_month_outlined),
        selectedIcon: Icon(Icons.calendar_month),
        label: 'Jadwal',
      ),
      NavigationDestination(
        icon: Icon(Icons.person_outline),
        selectedIcon: Icon(Icons.person),
        label: 'Profil',
      ),
    ],
  );
}

```

```
// ===== BARU (4) : tata letak layar lebar, rail di kiri + isi di kanan =====
Widget _tataLetakLebar() {
  return Row(
    children: [
      NavigationRail(
        selectedIndex: _indeksTerpilih,
        onDestinationSelected: _pilihTujuan,
        extended: true,
        leading: const Padding(
          padding: EdgeInsets.symmetric(vertical: 16),
          child: Icon(Icons.school, size: 32),
        ),
        destinations: const [
          NavigationRailDestination(
            icon: Icon(Icons.home_outlined),
            selectedIcon: Icon(Icons.home),
            label: Text('Beranda'),
          ),
          NavigationRailDestination(
            icon: Icon(Icons.calendar_month_outlined),
            selectedIcon: Icon(Icons.calendar_month),
            label: Text('Jadwal'),
          ),
          NavigationRailDestination(
            icon: Icon(Icons.person_outline),
            selectedIcon: Icon(Icons.person),
            label: Text('Profil'),
          ),
        ],
      ),
      const VerticalDivider(thickness: 1, width: 1),
      Expanded(child: _halaman[_indeksTerpilih]),
    ],
  );
}

// Drawer, isinya sama seperti subbagian sebelumnya.
Widget _buatDrawer() {
  return NavigationDrawer(
    selectedIndex: _indeksTerpilih,
    onDestinationSelected: (indeks) {
      _pilihTujuan(indeks);
    },
  );
}
```

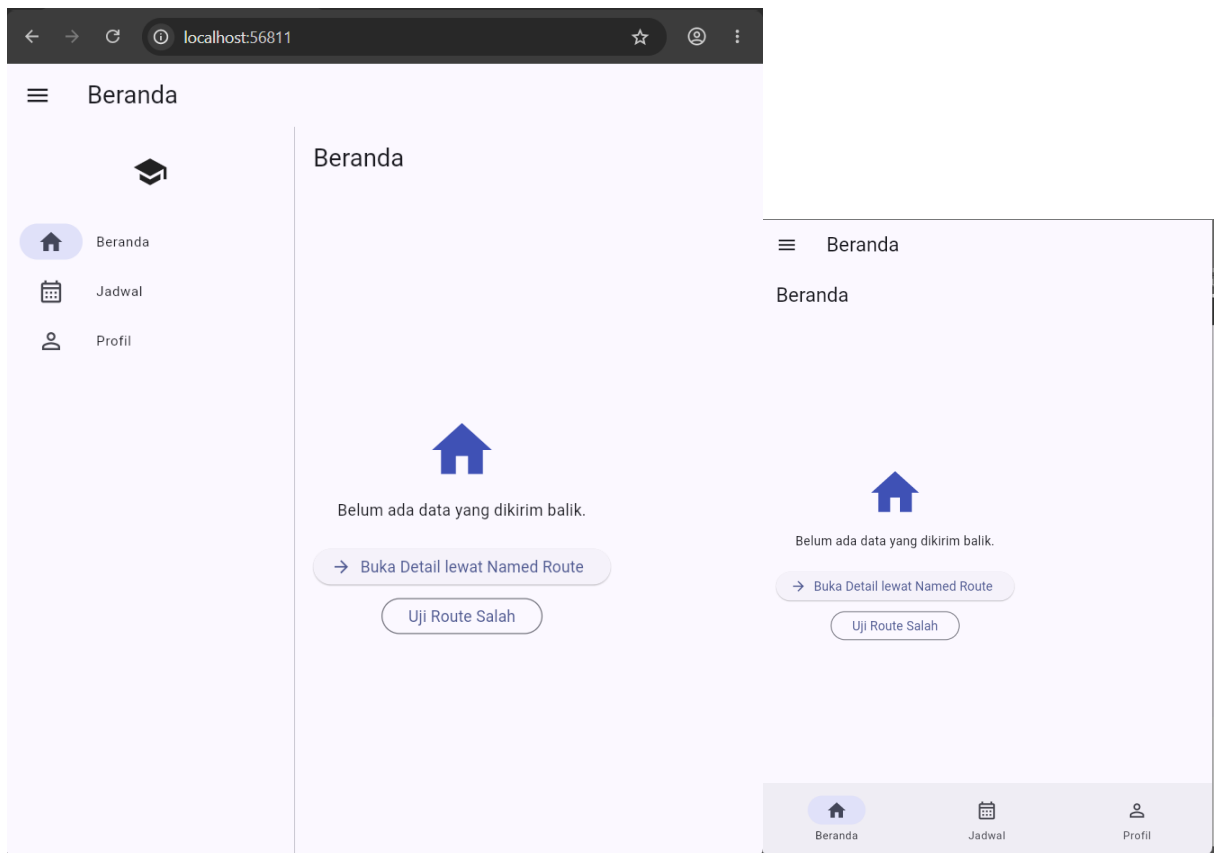
```

    Navigator.pop(context);
  },
  children: [
    const UserAccountsDrawerHeader(
      accountName: Text('Nama Mahasiswa'),
      accountEmail: Text('mahasiswa@kampus.ac.id'),
      currentAccountPicture: CircleAvatar(child: Icon(Icons.person)),
    ),
    const NavigationDrawerDestination(
      icon: Icon(Icons.home_outlined),
      selectedIcon: Icon(Icons.home),
      label: Text('Beranda'),
    ),
    const NavigationDrawerDestination(
      icon: Icon(Icons.calendar_month_outlined),
      selectedIcon: Icon(Icons.calendar_month),
      label: Text('Jadwal'),
    ),
    const NavigationDrawerDestination(
      icon: Icon(Icons.person_outline),
      selectedIcon: Icon(Icons.person),
      label: Text('Profil'),
    ),
    const Divider(indent: 28, endIndent: 28),
    ListTile(
      leading: const Icon(Icons.settings_outlined),
      title: const Text('Pengaturan'),
      onTap: () {
        Navigator.pop(context);
        Navigator.pushNamed(context, AppRoutes.pengaturan);
      },
    ),
  ],
);
}
}

```

- Properti `extended` bernilai `true` menampilkan label di samping ikon. Apabila bernilai `false`, hanya ikon yang tampil sehingga bilah menjadi lebih sempit. Properti `labelType` dapat digunakan untuk mengatur kemunculan label ketika `extended` bernilai `false`.
- Ringkasan isi berkas versi final ini adalah satu metode `build` beserta tiga metode pembantu, yaitu `_buatBilahBawah` untuk layar sempit, `_tataLetakLebar` untuk layar lebar, dan

_buatDrawer untuk panel samping. Keempatnya berada di dalam kelas _KerangkaNavigasiState.



Gambar 4.8 Perbandingan kerangka navigasi adaptif pada layar sempit dan layar lebar

i. Membuat Tab Navigation

Tab navigation dipakai untuk membagi isi satu halaman menjadi beberapa kelompok yang setara. Pada praktikum ini, halaman jadwal dibagi menjadi tiga *tab* berdasarkan hari kuliah. `DefaultTabController` digunakan agar `TabController` tidak perlu dibuat dan dilepas secara manual.

- Buka berkas `lib/pages/halaman_jadwal.dart`. Hapus seluruh isi lama, lalu salin kode berikut secara utuh.

```
// lib/pages/halaman_jadwal.dart
import 'package:flutter/material.dart';

class HalamanJadwal extends StatelessWidget {
  const HalamanJadwal({super.key});

  @override
  Widget build(BuildContext context) {
    return DefaultTabController(
      length: 3,
      child: Column(
        children: [
```

```

const TabBar(
  tabs: [
    Tab(icon: Icon(Icons.today), text: 'Senin'),
    Tab(icon: Icon(Icons.event), text: 'Rabu'),
    Tab(icon: Icon(Icons.event_available), text: 'Jumat'),
  ],
),
Expanded(
  child: TabBarView(
    children: [
      _buatDaftarJadwal(
        const ['Basis Data 07.00', 'Matematika Diskrit 10.00'],
      ),
      _buatDaftarJadwal(
        const ['PPB Lanjut 08.00', 'Jaringan 13.00'],
      ),
      _buatDaftarJadwal(
        const ['Kecerdasan Artifisial 09.00'],
      ),
    ],
  ),
),
],
),
);
}

Widget _buatDaftarJadwal(List<String> daftarJadwal) {
  return ListView.separated(
    itemCount: daftarJadwal.length,
    separatorBuilder: (context, indeks) => const Divider(height: 1),
    itemBuilder: (itemContext, indeks) {
      return ListTile(
        leading: const Icon(Icons.book_outlined),
        title: Text(daftarJadwal[indeks]),
        trailing: const Icon(Icons.chevron_right),
        onTap: () {
          Navigator.pushNamed(
            itemContext,
            '/detail',
            arguments: {

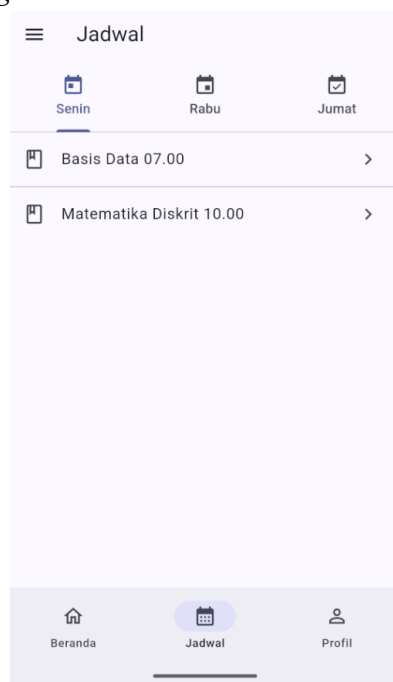
```

```

        'judul': daftarJadwal[indeks],
        'keterangan':
            'Rincian jadwal perkuliahan beserta ruang dan dosen pengampu.',
    },
    );
},
);
},
);
}
}
}

```

- Nilai properti `length` pada `DefaultTabController` harus sama dengan jumlah item pada `TabBar` dan jumlah anak pada `TabBarView`. Ketidaksesuaian ketiga nilai tersebut menyebabkan galat ketika *tab* dipindahkan.
- `TabBarView` memerlukan batasan tinggi yang jelas. Oleh karena itu *widget* tersebut dibungkus oleh `Expanded` ketika berada di dalam `Column`. Tanpa `Expanded` akan muncul galat berupa *unbounded height*.



Gambar 4.9 `TabBar` dan `TabBarView` pada halaman *jadwal* dengan *tab* *Senin* sebagai *tab* aktif

j. Ringkasan Perbedaan dan Panduan Memilih Navigasi

Subbab ini merupakan rangkuman agar mahasiswa dapat membedakan fungsi setiap konsep.

Tabel 4.2 Ringkasan perbedaan penggunaan dan pola program intinya

Materi	Ini untuk apa?	Pakai saat...	Jangan dipakai saat...	Contoh kode utama
Navigat or dan Route	Route adalah satu layar. Navigator seperti petugas yang mencatat layar mana yang sedang dibuka.	Saat mulai belajar perpindahan layar dan cara kerja tombol Kembali.	Ini adalah dasar pemahaman, bukan menu atau tombol yang langsung terlihat.	// Tentukan layar pertama MaterialApp(home: const HalamanBeranda(),);
Navigat or.push dan pop	push membuka layar baru. pop menutup layar itu lalu kembali ke layar sebelumnya.	Saat sebuah tombol hanya perlu membuka satu halaman baru, misalnya halaman Detail.	Jika halaman sangat banyak dan alamatnya ingin dikumpulkan di satu tempat, gunakan Named Routes.	// Buka halaman Detail Navigator.push(context, MaterialPageRoute(builder: () => const HalamanDetail(judul: 'Detail', keterangan: 'Contoh',),),); // Tutup halaman Detail Navigator.pop(context);
Named Routes	Setiap halaman diberi nama atau alamat, misalnya /profil. Alamat halaman dikumpulkan di satu tempat.	Saat aplikasi mempunyai banyak halaman atau satu halaman dibuka dari banyak tombol.	Untuk satu halaman sederhana yang hanya dibuka sekali, Navigator.push lebih mudah.	// Daftarkan alamat MaterialApp(routes: { '/profil': () => const HalamanProfil(), },); // Buka alamat /profil Navigator.pushNamed(context, '/profil',);
Navigati onBar	Menu yang selalu terlihat di bawah layar. Ketuk ikon untuk mengganti bagian utama aplikasi.	Saat aplikasi HP memiliki tiga sampai lima menu utama, misalnya Beranda, Jadwal, dan Profil.	Jangan dipakai untuk tombol Cari atau Tambah. Tombol seperti itu lebih cocok di BottomAppBar.	NavigationBar(selectedIndex: indeks, onDestinationSelected: (nomor) { setState(() => indeks = nomor); }, destinations: const [NavigationDestination(icon: Icon(Icons.home), label: 'Beranda',), NavigationDestination(icon: Icon(Icons.person), label: 'Profil',),],);
Bottom AppBar	Tempat berkumpulnya tombol kerja di bawah layar, misalnya Cari, Filter, dan Tambah.	Saat pengguna perlu melakukan pekerjaan pada halaman yang sedang dibuka.	Jangan dipakai sebagai menu untuk berpindah antara Beranda, Jadwal, dan Profil.	Scaffold(floatingActionButton: FloatingActionButton(onPressed: () {}, child: const Icon(Icons.add),), bottomNavigationBar: BottomAppBar(child: IconButton(onPressed: () {}, icon: const Icon(Icons.search),),),);

Materi	Ini untuk apa?	Pakai saat...	Jangan dipakai saat...	Contoh kode utama
				);
Navigati onDrawer	Menu yang muncul dari samping layar. Menu ini seperti laci yang dapat dibuka dan ditutup.	Saat menu cukup banyak atau ada menu tambahan seperti Pengaturan dan Bantuan.	Jika hanya ada tiga menu utama yang harus selalu terlihat, NavigationBar lebih mudah digunakan.	<pre>Scaffold(drawer: NavigationDrawer(onDestinationSelected: (nomor) { setState(() => indeks = nomor); Navigator.pop(context); }, children: const [NavigationDrawerDestination(icon: Icon(Icons.home), label: Text('Beranda'),),],),);</pre>
Navigati onRail	Menu yang berdiri di sebelah kiri layar. Bentuknya cocok untuk layar yang lebar.	Saat aplikasi dibuka di tablet, web, atau komputer dengan lebar sekitar 600 piksel atau lebih.	Jangan dipaksakan pada layar HP yang sempit. Pada HP, gunakan NavigationBar atau Drawer.	<pre>Row(children: [NavigationRail(selectedIndex: indeks, onDestinationSelected: (nomor) { setState(() => indeks = nomor); }, destinations: const [NavigationRailDestination(icon: Icon(Icons.home), label: Text('Beranda'),),],), const VerticalDivider(width: 1), Expanded(child: halaman[indeks]),],);</pre>
Tab Navigati on	Tab seperti sekat pada buku. Setiap tab menunjukkan kelompok isi yang berbeda, tetapi masih berada pada halaman yang sama.	Saat isi ingin dibagi menurut hari, jenis, atau status, misalnya Senin, Rabu, dan Jumat.	Jangan dipakai untuk membuka halaman baru yang membutuhkan tombol Kembali.	<pre>DefaultTabController(length: 3, child: Column(children: const [TabBar(tabs: [Tab(text: 'Senin'), Tab(text: 'Rabu'), Tab(text: 'Jumat'),],), Expanded(child: TabBarView(children: [Center(child: Text('Senin')), Center(child: Text('Rabu')), Center(child: Text('Jumat')),],),),],),);</pre>

F. Materi Praktikum dengan Prompt

Bagian ini membangun ulang Materi Praktikum dengan bantuan asisten AI. Hasil akhir yang dituju tetap sama, tetapi kode diperoleh melalui *prompt* bertahap. Tujuannya bukan menggantikan proses belajar, melainkan melatih mahasiswa menulis kebutuhan dengan jelas, membaca hasil AI, menemukan kesalahan, dan menjelaskan kembali cara kerja kode. Setiap kode dari AI harus dibaca sebelum disalin, dijalankan, dibandingkan dengan ketentuan pada materi sebelumnya, dan diuji. Kode yang belum dapat dijelaskan mahasiswa tidak boleh dianggap selesai.

Tabel 5.1 Unsur prompt yang digunakan pada praktikum berbantuan AI

Unsur prompt	Penjelasan
Peran	Menentukan siapa AI dan tingkat pembacanya, misalnya asisten Flutter untuk mahasiswa pemula.
Konteks	Menjelaskan nama proyek, materi, berkas yang sudah ada, dan tahap yang sedang dikerjakan.
Tugas	Menyebutkan satu hasil yang jelas, misalnya menulis ulang satu berkas tertentu.
Batasan	Menjaga hasil tetap sesuai materi, paket yang diizinkan, nama kelas, dan struktur folder.
Format keluaran	Menentukan apakah AI harus memberi rencana, kode lengkap, bagian yang berubah, atau penjelasan.
Verifikasi	Meminta AI menjelaskan alasan kode dan memberi daftar pemeriksaan yang dapat diuji mahasiswa.

a. Menyiapkan Proyek Praktikum dengan Bantuan AI

Tahap ini menghasilkan proyek baru yang terpisah dari praktikum manual. Dengan demikian, hasil kerja biasa dan hasil berbantuan AI dapat dibandingkan.

- Buka terminal Visual Studio Code, lalu jalankan perintah berikut. Ganti nim dengan nomor induk mahasiswa masing-masing.

```
flutter create navigasi_flutter_ai_nim
cd navigasi_flutter_ai_nim
code .
```

- Setelah proyek terbuka, gunakan percakapan AI baru agar konteks dari proyek lain tidak ikut terbawa.
- Ketik *prompt* konteks awal berikut pada panel percakapan AI.

Kamu berperan sebagai asisten pemrograman Flutter untuk mahasiswa pemula.

Konteks: saya membuat proyek bernama navigasi_flutter_ai_nim di Visual Studio Code. Materi yang dipelajari adalah Navigator, route, named routes, NavigationBar, BottomAppBar, NavigationDrawer, NavigationRail, dan Tab Navigation.

Tujuan akhir: membantu saya membuat aplikasi Navigasi Kampus Digital dengan menu Beranda, Jadwal, dan Profil. Aplikasi juga mempunyai halaman Detail, Pengaturan, dan Catatan.

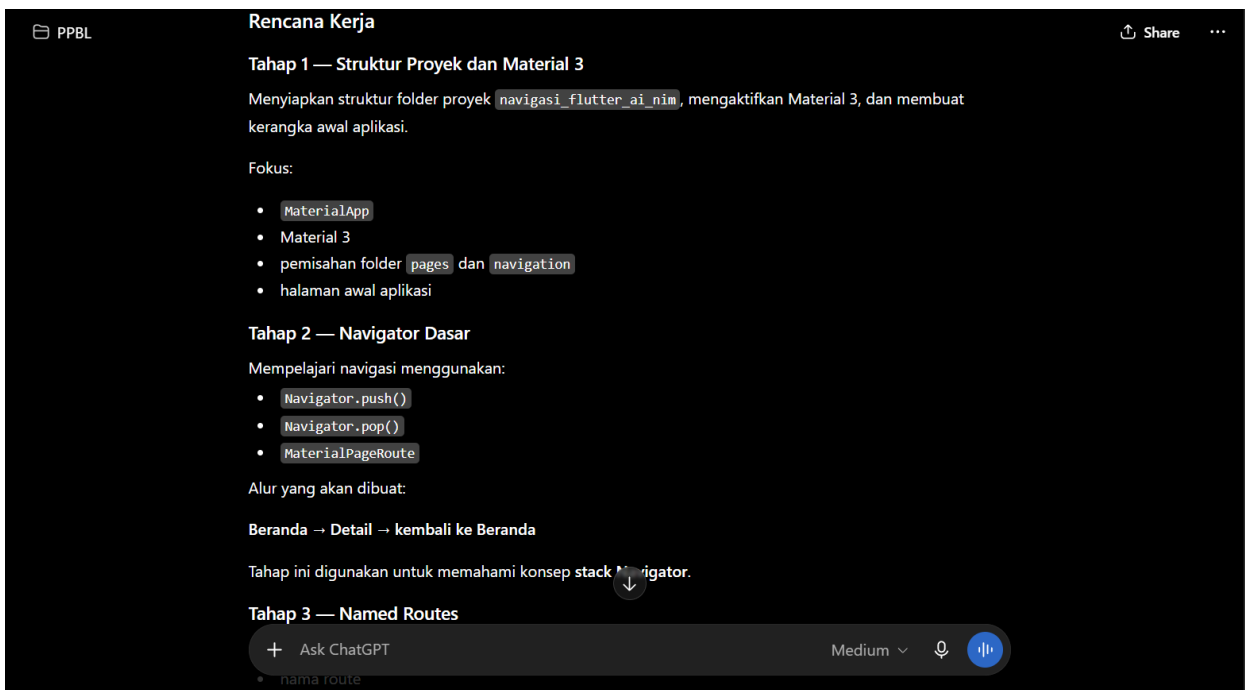
Batasan:

1. Gunakan Flutter Material 3 dan hanya package:flutter/material.dart.
2. Jangan memakai paket tambahan atau state management eksternal.
3. Pisahkan berkas halaman di lib/pages dan pengaturan route di lib/navigation.

4. Kerjakan bertahap. Jangan menulis semua kode sekaligus.
5. Setiap kode harus lengkap satu berkas, tidak boleh memakai tanda "..." untuk menghilangkan bagian kode.
6. Gunakan bahasa Indonesia dan nama kelas yang sama dengan nama berkas.

Format keluaran: tuliskan rencana kerja dan daftar berkas yang akan dibuat. Jangan menulis kode sebelum saya meminta tahap tertentu.

- Bandingkan rencana AI dengan struktur yang dipelajari pada materi a-i. Rencana sekurang-kurangnya memuat folder pages, folder navigation, dan berkas main.dart.
- Apabila AI langsung memberikan semua kode, jangan menyalinnya. Kirim ulang kalimat: Ikuti format keluaran. Berikan rencana dan daftar berkas saja, tanpa kode.
- Hasil yang harus diperiksa: Proyek dapat dibuka, folder lib terlihat, dan AI belum menulis kode sebelum diminta.



Gambar 5.1 Rencana Kerja yang dihasilkan oleh AI

b. Memahami Navigator dan Routes dengan Bantuan AI

Tahap ini sama dengan materi konsep Navigator dan Routes. AI dipakai untuk memberikan analogi, lalu menghasilkan halaman pertama yang sederhana.

- Buat folder lib/pages dan lib/navigation secara manual agar struktur proyek tetap dipahami.
- Kirim *prompt* berikut dalam percakapan AI yang sama.

Lanjutkan ke tahap konsep Navigator dan Route.

Tugas:

1. Jelaskan Route sebagai satu layar dan Navigator sebagai pengelola tumpukan layar. Gunakan analogi tumpukan buku dan maksimal enam poin.

2. Buat isi lengkap `lib/pages/halaman_beranda.dart`. Gunakan `StatelessWidget` bernama `HalamanBeranda`, `Scaffold`, `AppBar` berjudul 'Beranda', ikon home, dan teks 'Halaman Beranda'.
3. Buat isi lengkap `lib/main.dart`. Fungsi `main` menjalankan `AplikasiNavigasi`. `MaterialApp` memakai `Material 3`, `colorSchemeSeed` indigo, banner debug dimatikan, dan home menuju `HalamanBeranda`.

Batasan: hanya gunakan `package:flutter/material.dart`. Tampilkan setiap berkas dalam blok kode terpisah dan tulis nama berkas sebelum blok kode.

Setelah kode, jelaskan fungsi `MaterialApp`, `Scaffold`, `BuildContext`, `Route`, dan `Navigator` dengan bahasa sederhana.

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan *hot restart*.
- Periksa bahwa `MaterialApp` berada di atas `HalamanBeranda`. `BuildContext` yang dipakai untuk navigasi harus berada di bawah `MaterialApp`.
- Hasil yang harus diperiksa: Aplikasi menampilkan `AppBar` Beranda, ikon rumah, dan teks Halaman Beranda tanpa galat.



Gambar 5.2 Konsep Navigator dan Route dengan AI sebagai Tools

c. Menerapkan `Navigator.push` dan `Navigator.pop` dengan Bantuan AI

Pada tahap ini AI diminta membuat perpindahan halaman secara langsung. Hasilnya sama dengan praktikum manual: halaman Detail dibuka dengan push dan dapat mengirim nilai kembali melalui pop.

- Buat berkas kosong `lib/pages/halaman_detail.dart`, kemudian kirim *prompt* berikut.

Lanjutkan proyek `navigasi_flutter_ai_nim` dengan `Navigator.push` dan `Navigator.pop`.

Tugas 1 - `lib/pages/halaman_detail.dart`:

1. Buat `StatelessWidget` bernama `HalamanDetail`.

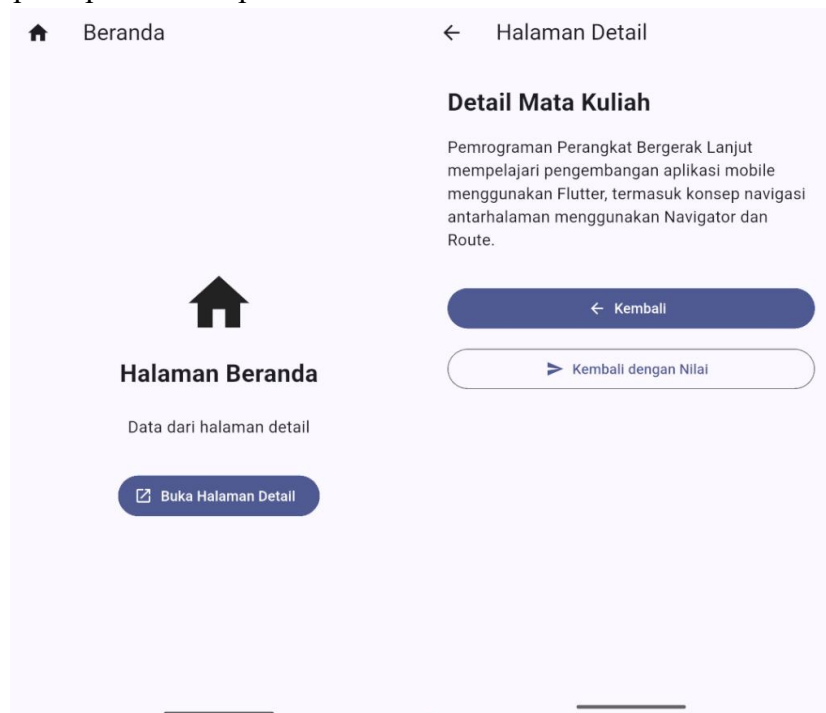
2. Terima parameter wajib judul dan keterangan bertipe String.
3. Tampilkan Scaffold dengan AppBar, judul, keterangan, tombol Kembali, dan tombol Kembali dengan Nilai.
4. Tombol pertama memanggil Navigator.pop(context).
5. Tombol kedua memanggil Navigator.pop(context, 'Data dari halaman detail').

Tugas 2 - lib/pages/halaman_beranda.dart:

1. Tulis ulang seluruh berkas menjadi StatefulWidget.
2. Sediakan variabel _pesan dengan teks awal 'Belum ada data yang dikirim balik.'.
3. Buat fungsi async _bukaDetail yang memakai await Navigator.push<String> dan MaterialPageRoute.
4. Kirim judul 'Detail Mata Kuliah' dan keterangan tentang Pemrograman Perangkat Bergerak Lanjut.
5. Setelah route ditutup, periksa mounted, lalu tampilkan nilai hasil melalui setState.
6. Sediakan tombol 'Buka Halaman Detail'.

Tampilkan dua berkas secara lengkap dan terpisah. Setelah kode, jelaskan urutan push, await, pop, nilai kembali, mounted, dan setState.

- Salin kedua berkas, jalankan aplikasi, lalu coba tombol Kembali dan Kembali dengan Nilai.
- Jika tombol kembali otomatis tidak muncul pada AppBar Detail, minta AI memeriksa apakah halaman benar-benar dibuka dengan Navigator.push, bukan pushReplacement.
- Hasil yang harus diperiksa: Halaman Detail terbuka di atas Beranda, dapat ditutup, dan nilai dari pop tampil kembali pada Beranda.



Gambar 5.3 Konsep Navigator.push dan Navigator.pop dengan AI sebagai Tools

d. Menerapkan Named Routes dengan Bantuan AI

Tahap ini memusatkan nama *route* agar halaman tidak saling mengimpor secara langsung. AI harus menghasilkan AppRoutes, memperbarui main.dart, dan mengganti pemanggilan push menjadi pushNamed.

- Buat berkas kosong lib/navigation/app_routes.dart, lalu kirim *prompt* berikut.

Ubah proyek agar memakai named routes.

Tugas 1 - lib/navigation/app_routes.dart:

1. Buat class AppRoutes.
2. Buat konstanta beranda = '/' dan detail = '/detail'.
3. Buat daftarRoute() untuk route beranda.
4. Buat bentukRoute(RouteSettings settings) untuk route detail yang membaca arguments Map<String, String> lalu meneruskannya ke HalamanDetail.
5. Buat routeTidakDikenal() yang menampilkan halaman 'Route Tidak Ditemukan'.

Tugas 2 - lib/main.dart:

Gunakan initialRoute, routes, onGenerateRoute, dan onUnknownRoute dari AppRoutes. Pertahankan tema Material 3 berwarna indigo.

Tugas 3 - lib/pages/halaman_beranda.dart:

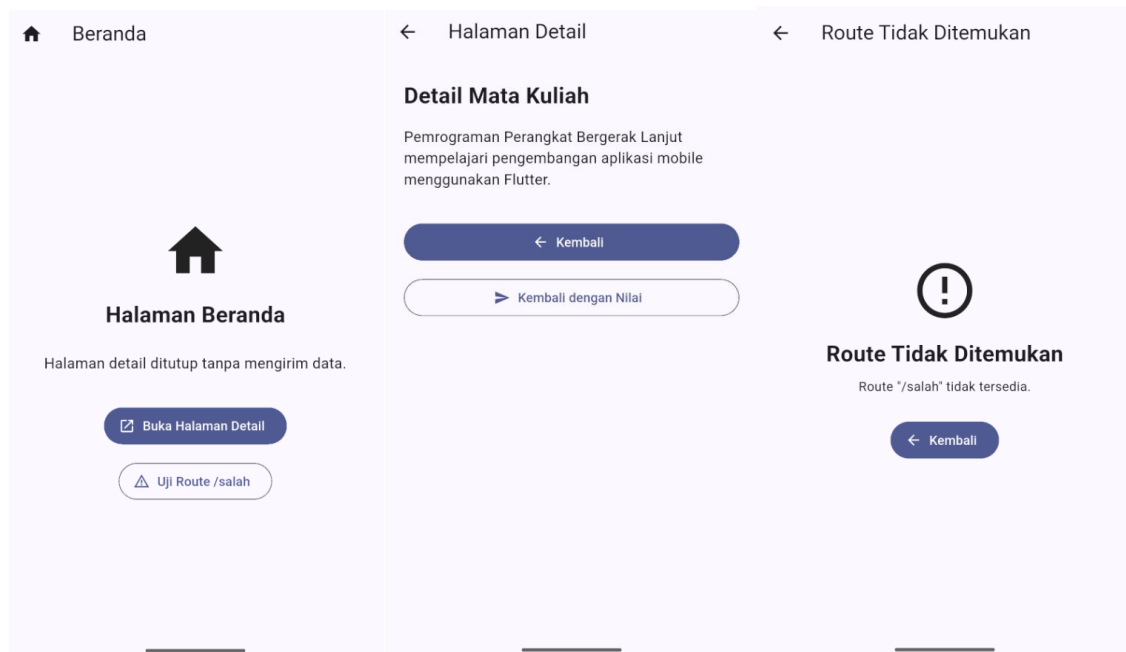
Tulis ulang pemanggilan halaman Detail memakai Navigator.pushNamed, AppRoutes.detail, serta arguments berisi judul dan keterangan. Tambahkan tombol untuk menguji route '/salah'.

Batasan:

- Tampilkan tiga berkas secara lengkap.
- Jangan mendaftarkan halaman yang berkasnya belum dibuat.
- Jangan memakai tanda "..." untuk menyingkat kode.

Setelah kode, buat tabel perbedaan Navigator.push dan Navigator.pushNamed dengan bahasa mahasiswa pemula.

- Jalankan tombol Detail dan tombol uji *route* salah. Pastikan *route* salah ditangani oleh onUnknownRoute, bukan membuat aplikasi berhenti.
- Cocokkan setiap nama *route*. Satu karakter yang berbeda dapat menyebabkan *route* tidak ditemukan saat aplikasi dijalankan.
- Hasil yang harus diperiksa: Halaman Detail terbuka melalui nama *route*, arguments diterima, dan *route* yang tidak terdaftar menampilkan halaman penanganan galat.



Gambar 5.4 Menerapkan Named Routes dengan AI sebagai Tools

e. Membuat NavigationBar dengan Bantuan AI

NavigationBar mengganti tujuan utama di dalam satu Scaffold dan tidak menambah tumpukan *route*.

- Buat berkas kosong `halaman_jadwal.dart`, `halaman_profil.dart`, `halaman_pengaturan.dart`, dan `kerangka_navigasi.dart` sesuai foldernya.
- Kirim *prompt* berikut. Minta AI menjawab satu berkas demi satu berkas apabila jawabannya terlalu panjang.

Lanjutkan proyek dengan NavigationBar Material 3.

Tulis kode lengkap untuk berkas berikut.

1. `lib/pages/halaman_beranda.dart`
 - Pertahankan `StatefulWidget`, `_pesan`, dan tombol yang membuka route `Detail`.
 - Hapus `Scaffold` dan `AppBar` dari halaman ini karena keduanya akan disediakan oleh `KerangkaNavigasi`.
 - Kembalikan langsung isi berupa `Padding` atau `Center`.
2. `lib/pages/halaman_jadwal.dart`
 - `StatelessWidget` bernama `HalamanJadwal`.
 - Versi sederhana berisi `Center` dan teks 'Halaman Jadwal'.
 - Jangan memakai `Scaffold` karena halaman akan berada di dalam kerangka navigasi.
3. `lib/pages/halaman_profil.dart`
 - `StatelessWidget` bernama `HalamanProfil`.
 - Versi sederhana berisi `Center` dan teks 'Halaman Profil'.
 - Jangan memakai `Scaffold`.
4. `lib/pages/halaman_pengaturan.dart`

- StatelessWidget bernama HalamanPengaturan.
- Gunakan Scaffold dan AppBar karena halaman dibuka sebagai route penuh.

5. lib/navigation/kerangka_navigasi.dart

- StatefulWidget bernama KerangkaNavigasi.
- Simpan _indeksTerpilih.
- Daftar halaman: HalamanBeranda, HalamanJadwal, dan HalamanProfil.
- Scaffold memiliki AppBar, body sesuai indeks, dan NavigationBar.
- Buat tiga NavigationDestination: Beranda, Jadwal, dan Profil, masing-masing memiliki icon dan selectedIcon.

6. lib/navigation/app_routes.dart

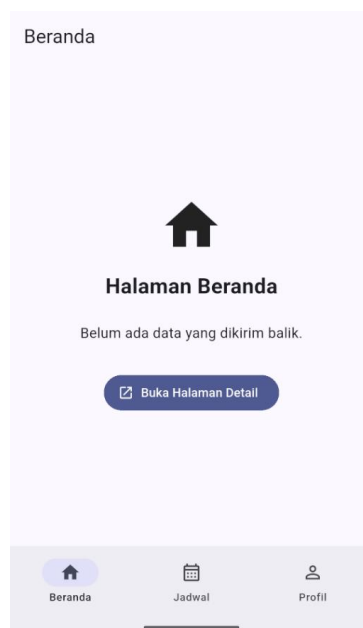
- Tulis ulang versi lengkap.
- Route '/' membuka KerangkaNavigasi.
- Daftarkan '/pengaturan'.
- Pertahankan route '/detail', onGenerateRoute, dan penanganan route tidak dikenal.

7. lib/main.dart

- Pastikan initialRoute memakai AppRoutes.beranda dan seluruh konfigurasi route tetap terhubung.

Batasan: jumlah halaman utama, judul, dan NavigationDestination harus sama-sama tiga. Jangan membuat Scaffold kedua pada HalamanBeranda, HalamanJadwal, dan HalamanProfil. Jelaskan alur indeks terpilih setelah kode.

- Salin setiap kode ke berkas yang tepat. Jika AI memberikan beberapa berkas dalam satu blok, minta AI memisahnya agar tidak salah salin.
- Periksa tiga jumlah berikut: tiga halaman, tiga judul, dan tiga NavigationDestination. Ketiganya harus sama.
- Hasil yang harus diperiksa: NavigationBar tampil pada bagian bawah dan setiap tujuan mengganti isi tanpa menambah tombol Kembali.



Gambar 5.5 Membuat NavigationBar dengan AI sebagai Tools

f. Membuat BottomAppBar dengan Bantuan AI

BottomAppBar digunakan untuk menempatkan tombol aksi pada halaman aktif. *Widget* ini tidak menggantikan NavigationBar sebagai pemilih tujuan utama.

- Buat berkas kosong lib/pages/halaman_catatan.dart. KerangkaNavigasi tidak diubah pada tahap ini.

Tambahkan contoh BottomAppBar tanpa menghapus NavigationBar yang sudah ada.

Tugas 1 - lib/pages/halaman_catatan.dart:

1. Buat StatelessWidget bernama HalamanCatatan dengan Scaffold dan AppBar berjudul 'Catatan'.
2. Body menampilkan teks 'Daftar catatan kuliah'.
3. Tambahkan FloatingActionButton berikan add pada posisi centerDocked.
4. Gunakan BottomAppBar dengan CircularNotchedRectangle dan notchMargin 8.
5. Isi BottomAppBar dengan tombol Search, Filter, Sort, dan More. Sisakan ruang di tengah agar tombol tidak tertutup FloatingActionButton.
6. Saat FloatingActionButton ditekan, tampilkan SnackBar 'Tambah catatan baru'.

Tugas 2 - lib/navigation/app_routes.dart:

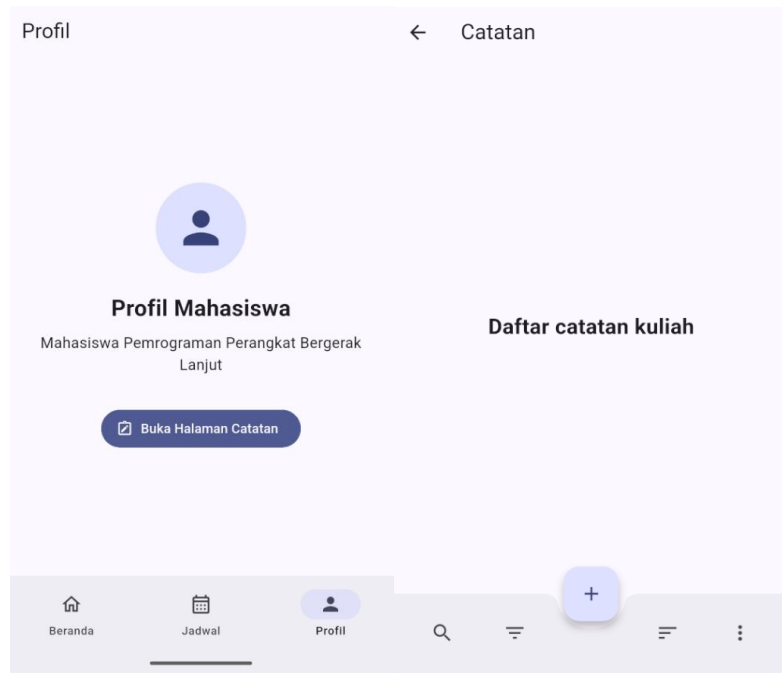
Tampilkan hanya tiga bagian yang perlu ditambahkan: import halaman_catatan.dart, konstanta catatan = '/catatan', dan pendaftaran route catatan pada daftarRoute().

Tugas 3 - lib/pages/halaman_profil.dart:

Tulis ulang seluruh berkas tanpa Scaffold. Tampilkan profil sederhana dan tombol 'Buka Halaman Catatan' yang memanggil Navigator.pushNamed dengan AppRoutes.catatan.

Setelah kode, jelaskan perbedaan NavigationBar dan BottomAppBar dalam empat kalimat sederhana.

- Pastikan BottomAppBar hanya muncul pada HalamanCatatan. NavigationBar utama pada KerangkaNavigasi harus tetap ada.
- Uji tombol Buka Halaman Catatan, tombol aksi pada BottomAppBar, FloatingActionButton, dan tombol Kembali.
- Hasil yang harus diperiksa: Halaman Catatan terbuka sebagai *route* baru, menampilkan BottomAppBar dan FloatingActionButton, lalu dapat kembali ke Profil.



Gambar 5.6 Membuat BottomAppBar dengan AI sebagai Tools

g. Membuat NavigationDrawer dengan Bantuan AI

Drawer menyediakan menu dari sisi layar. Pada proyek ini, pilihan Beranda, Jadwal, dan Profil mengganti indeks, sedangkan Pengaturan membuka *route* baru.

- Kirim *prompt* berikut untuk memperbarui kerangka navigasi yang sudah ada.

Tulis ulang seluruh isi `lib/navigation/kerangka_navigasi.dart` dengan menambahkan `NavigationDrawer` tanpa menghilangkan `NavigationBar`.

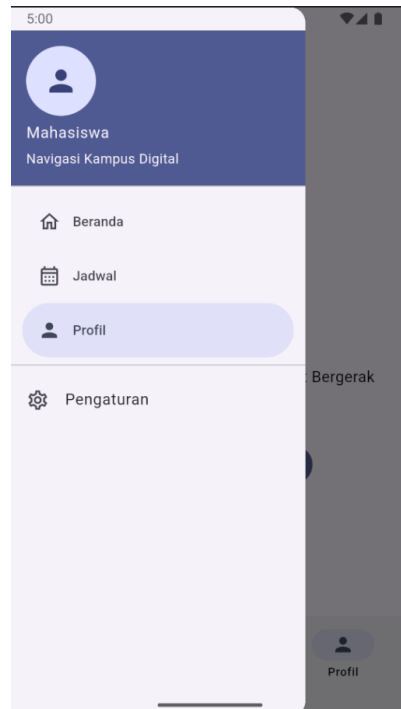
Ketentuan:

1. Pertahankan `_indeksTerpilih`, daftar halaman, daftar judul, dan fungsi `_pilihTujuan`.
2. Tambahkan drawer: `_buatDrawer()` pada `Scaffold`, tepat setelah `appBar`.
3. Buat metode `_buatDrawer` di dalam class `_KerangkaNavigasiState`, setelah metode `build` dan sebelum kurung penutup class.
4. `NavigationDrawer` berisi `UserAccountsDrawerHeader`, tujuan Beranda, Jadwal, Profil, `Divider`, dan `ListTile` Pengaturan.
5. Saat tujuan utama dipilih, ubah indeks lalu tutup drawer dengan `Navigator.pop(context)`.
6. Saat Pengaturan dipilih, tutup drawer dahulu, kemudian buka `AppRoutes.pengaturan` dengan `Navigator.pushNamed`.
7. Gunakan icon dan `selectedIcon` pada setiap `NavigationDrawerDestination`.
8. Jangan menghapus `NavigationBar` yang sudah berfungsi.

Setelah kode, beri komentar BARU hanya pada baris drawer di `Scaffold` dan pada metode `_buatDrawer`. Jelaskan mengapa `Navigator.pop` digunakan sebelum membuka Pengaturan.

- Bandingkan kode baru dengan versi sebelumnya. Perubahan utama seharusnya hanya properti *drawer* dan satu metode `_buatDrawer`.
- Periksa bahwa `_buatDrawer` masih berada di dalam class `_KerangkaNavigasiState` agar dapat memakai `_indeksTerpilih` dan `context`.

- Hasil yang harus diperiksa: Ikon menu membuka *Drawer*, tiga tujuan utama mengganti isi, dan menu Pengaturan membuka halaman baru setelah *Drawer* tertutup.



Gambar 5.7 Membuat *NavigationDrawer* dengan AI sebagai Tools

h. Membuat *NavigationRail* Adaptif dengan Bantuan AI

NavigationRail dipakai ketika layar cukup lebar. AI diminta membuat satu kerangka yang menampilkan *NavigationBar* pada layar sempit dan *NavigationRail* pada layar lebar.

- Buka proyek pada Chrome atau desktop agar perubahan lebar layar mudah diuji, lalu kirim *prompt* berikut.

Ubah lib/navigation/kerangka_navigasi.dart menjadi tata letak adaptif dan tampilkan seluruh isi berkas.

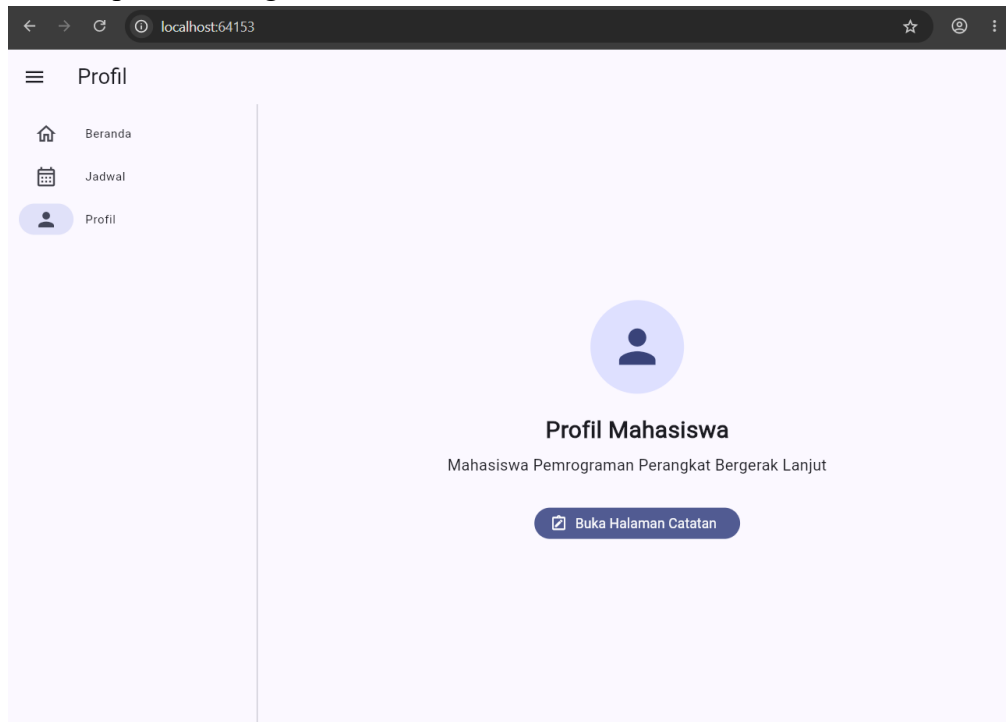
Ketentuan:

1. Baca lebar layar dengan `MediaQuery.of(context).size.width`.
2. Anggap layar lebar jika lebarnya 600 piksel atau lebih.
3. Pada layar kurang dari 600, tampilkan isi halaman dan *NavigationBar* di bagian bawah.
4. Pada layar 600 atau lebih, sembunyikan *NavigationBar* dan tampilkan Row berisi *NavigationRail*, *VerticalDivider*, dan *Expanded* untuk isi halaman.
5. *NavigationRail* memakai `selectedIndex`, `onDestinationSelected`, tiga tujuan yang sama, dan `extended: true`.
6. Pertahankan *NavigationDrawer* dan route Pengaturan.
7. Buat metode `_buatBilahBawah`, `_tataLetakLebar`, dan `_buatDrawer` di dalam `_KerangkaNavigasiState`.
8. Jangan membuat dua daftar halaman yang berbeda dan jangan menampilkan *NavigationBar* serta *NavigationRail* secara bersamaan.

Setelah kode, jelaskan alur keputusan lebar layar dalam bentuk: jika kondisi, widget yang tampil, dan widget yang disembunyikan.

- Jalankan aplikasi pada Chrome, lalu ubah lebar jendela melewati batas 600 piksel.

- Jika muncul galat `unbounded width` atau `overflow`, minta AI memeriksa posisi `Expanded` setelah `VerticalDivider`.
- Hasil yang harus diperiksa: Layar sempit menampilkan `NavigationBar`, sedangkan layar lebar menampilkan `NavigationRail` dan isi halaman di sebelah kanan.



Gambar 5.8 Membuat `NavigationRail` dengan AI sebagai Tools

i. Membuat Tab Navigation dengan Bantuan AI

Tahap terakhir mengubah `HalamanJadwal` menjadi tiga *tab*. *Tab* hanya mengganti isi di dalam halaman `Jadwal`, sedangkan item `jadwal` tetap dapat membuka *route* `Detail`.

- Kirim *prompt* berikut untuk mengganti versi sederhana halaman `jadwal`.

Tulis ulang seluruh isi `lib/pages/halaman_jadwal.dart` menggunakan `Tab Navigation`.

Ketentuan:

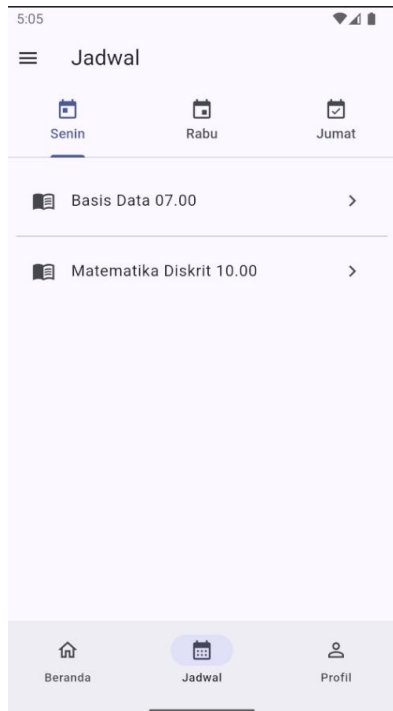
1. Buat `StatelessWidget` bernama `HalamanJadwal` tanpa `Scaffold` dan tanpa `AppBar` karena keduanya sudah dimiliki `KerangkaNavigasi`.
2. Gunakan `DefaultTabController` dengan `length: 3`.
3. Gunakan `Column` yang berisi `TabBar` dan `Expanded` yang membungkus `TabBarView`.
4. Buat tiga *tab*: `Senin` dengan `Icons.today`, `Rabu` dengan `Icons.event`, dan `Jumat` dengan `Icons.event_available`.
5. Isi `Senin`: `'Basis Data 07.00'` dan `'Matematika Diskrit 10.00'`.
6. Isi `Rabu`: `'PPB Lanjut 08.00'` dan `'Jaringan 13.00'`.
7. Isi `Jumat`: `'Kecerdasan Artifisial 09.00'`.
8. Buat metode `_buatDaftarJadwal(List<String> daftarJadwal)` yang mengembalikan `ListView.separated`.
9. Setiap `ListTile` memiliki ikon buku, judul, ikon `chevron`, dan `onTap`.
10. `onTap` memanggil `Navigator.pushNamed` ke `'/detail'` dengan arguments judul dan keterangan.

Batasan:

- Jumlah `length`, jumlah Tab, dan jumlah anak `TabBarView` harus sama-sama tiga.
- Semua teks jadwal harus berada di dalam tanda petik.
- Jangan menyingkat kode dengan tanda "...".

Setelah kode, jelaskan mengapa `TabBarView` harus dibungkus `Expanded` dan apa perbedaan `Tab Navigation` dengan `Navigator.pushNamed`.

- Salin kode, jalankan aplikasi, lalu buka menu Jadwal. Uji *tab* Senin, Rabu, dan Jumat.
- Ketuk satu jadwal dan pastikan *route* Detail terbuka. Jika *route* tidak ditemukan, periksa kembali `AppRoutes` dari tahap *Named Routes*.
- Hasil yang harus diperiksa: Ketiga *tab* dapat dipindahkan tanpa galat, setiap *tab* menampilkan data yang tepat, dan item jadwal membuka halaman Detail.



Gambar 5.9 Membuat Tab Navigation dengan AI sebagai Tools

j. Prompt untuk Menelusuri Galat

Apabila aplikasi belum berjalan, baca pesan galat lebih dahulu. Catat nama berkas dan nomor baris, kemudian kirim *prompt* berikut. Jangan meminta AI menulis ulang semua berkas karena perubahan akan sulit dilacak.

Saya mendapatkan galat berikut saat menjalankan proyek Flutter navigasi_flutter_ai_nim:

<tempelkan pesan galat lengkap di sini>

Tugas kamu:

1. Jelaskan arti galat dengan bahasa sederhana.
2. Sebutkan nama berkas dan baris yang paling mungkin menjadi penyebab.
3. Tampilkan perbaikan minimal, hanya bagian kode yang berubah.
4. Jelaskan mengapa perbaikan tersebut menyelesaikan galat.

5. Berikan satu langkah pengujian setelah perbaikan.

Jangan menulis ulang seluruh berkas dan jangan mengubah fitur lain.

- Terapkan perbaikan secara manual. Jalankan flutter analyze terlebih dahulu, kemudian flutter run.

k. Menguji Aplikasi dan Menguji Pemahaman

- Uji Navigator.push dan pop, termasuk nilai yang dikirim kembali dari halaman Detail.
- Uji *named route*, arguments, dan halaman untuk *route* yang tidak terdaftar.
- Pada layar sempit, uji NavigationBar dan Drawer. Pada layar lebar, uji NavigationRail.
- Buka HalamanCatatan untuk menguji BottomAppBar dan FloatingActionButton.
- Buka Jadwal, pindahkan ketiga *tab*, lalu ketuk item jadwal untuk membuka Detail.

Setelah semua fitur berjalan, tutup panel AI dan jawab tanpa bantuan: kapan memakai push dibanding pushNamed; mengapa NavigationBar tidak menambah *route*; mengapa BottomAppBar bukan menu tujuan; kapan NavigationRail muncul; serta mengapa TabBarView dibungkus Expanded.

- Jika ada kode yang belum dipahami, gunakan *prompt* penjelasan berikut tanpa meminta AI mengubah kodenya.

Jelaskan potongan kode Flutter berikut baris demi baris untuk mahasiswa pemula. Jangan mengubah kodenya.

<tempelkan potongan kode yang belum dipahami>

Untuk setiap bagian, jelaskan:

1. Apa tugasnya.
2. Data apa yang masuk dan keluar.
3. Apa yang terjadi saat pengguna menekan tombol.
4. Galat apa yang mungkin muncul jika bagian itu dihapus.

Akhiri dengan dua pertanyaan singkat untuk menguji pemahaman saya.

G. Tugas Praktikum

Soal 1: Aplikasi Layanan Warga Smart City “Nusantara Cerdas Mobile”

Dasbor Nusantara Cerdas yang dibangun pada praktikum sebelumnya dinilai berhasil oleh Pemerintah Kota Nusantara. Karena hanya berupa satu halaman, dasbor tersebut kini terasa sesak. Dinas Komunikasi dan Informatika memutuskan mengembangkannya menjadi aplikasi layanan warga yang terdiri atas banyak halaman, agar warga tidak lagi menggulir satu halaman panjang untuk menemukan layanan yang dicari. Kepala dinas menjelaskan bahwa aplikasi baru harus memiliki tiga layanan utama yang dapat dijangkau langsung dari mana saja, yaitu Beranda yang memuat ringkasan enam pilar smart city, Layanan yang memuat daftar layanan publik, dan Warga yang memuat identitas serta riwayat laporan pengguna. Ketiga layanan tersebut bukan halaman baru yang ditumpuk, melainkan tujuan utama yang saling menggantikan isi layar. Petugas balai

kota menegaskan bahwa menekan tombol kembali perangkat saat berpindah antar tujuan tidak boleh menutup aplikasi secara tiba-tiba.

Selain tiga layanan utama, terdapat pula menu pendukung yang jarang dibuka, yaitu Pengaturan Kota, Tentang Aplikasi, dan Keluar. Menu pendukung ini diminta ditempatkan pada panel samping agar tidak memenuhi bagian bawah. Sementara itu, daftar layanan publik pada tujuan Layanan perlu dikelompokkan menurut bidang, yaitu Perizinan, Kesehatan, dan Transportasi, dan warga dapat berpindah antarkelompok dengan menggeser layar tanpa meninggalkan halaman Layanan. Setiap layanan pada daftar dapat ditekan untuk membuka halaman rincian yang memuat nama layanan, dinas penanggung jawab, jam operasional, dan keterangan singkat. Halaman rincian dibuka sebagai halaman baru dan memiliki tombol kembali. Pada halaman rincian tersedia tombol Ajukan Permohonan yang menutup halaman sekaligus mengirim pesan hasil pengajuan kembali ke daftar layanan, misalnya Permohonan izin usaha telah diajukan, dan pesan tersebut ditampilkan sebagai pemberitahuan singkat. Kepala dinas menambahkan satu permintaan terakhir. Aplikasi tidak hanya dipakai pada telepon genggam petugas lapangan, tetapi juga pada layar sentuh berukuran besar di lobi balai kota. Oleh sebab itu, ketika aplikasi dibuka pada layar lebar, kerangka navigasi harus berpindah dari bilah bawah menjadi bilah samping agar ruang layar termanfaatkan dengan baik. Sebagai anggota tim pengembang, tugas Anda adalah mewujudkan purwarupa aplikasi tersebut menggunakan Flutter.

Ketentuan Pengerjaan

- Buat proyek Flutter baru dengan nama `nusantara_cerdas_nav_nim`, lalu susun berkas ke dalam folder `lib/pages` untuk halaman dan `lib/navigation` untuk pengaturan navigasi.
- Buat berkas `lib/navigation/app_routes.dart` yang memuat seluruh konstanta nama *route*, metode `daftarRoute` untuk properti *routes*, metode `bentukRoute` untuk `onGenerateRoute`, dan metode `routeTidakDikenal` untuk `onUnknownRoute`. Seluruh perpindahan halaman wajib memakai *named route*, bukan penyebutan nama kelas halaman secara langsung.
- Buat berkas `lib/navigation/kerangka_navigasi.dart` berupa `StatefulWidget` bernama `KerangkaNavigasi` yang menyimpan indeks tujuan aktif, kemudian pasang *route* beranda pada `MaterialApp` agar mengarah ke kerangka navigasi tersebut.
- Terapkan `NavigationBar` Material 3 dengan tiga tujuan utama, yaitu Beranda, Layanan, dan Warga, lengkap dengan icon dan `selectedIcon`. Berpindah tujuan tidak boleh menambah tumpukan *route*.
- Terapkan `NavigationDrawer` yang memuat tiga tujuan utama yang sama beserta menu pendukung Pengaturan Kota, Tentang Aplikasi, dan Keluar. Menu pendukung dibuka melalui `Navigator.pushNamed`, dan *drawer* wajib ditutup terlebih dahulu sebelum halaman baru dibuka.
- Terapkan *tab navigation* pada halaman Layanan menggunakan `DefaultTabController`, `TabBar`, dan `TabBarView` dengan tiga *tab*, yaitu Perizinan, Kesehatan, dan Transportasi. Setiap *tab* menampilkan minimal tiga layanan dalam bentuk daftar.
- Buat halaman rincian layanan yang menerima data melalui `arguments` pada *named route*, kemudian tampilkan nama layanan, dinas penanggung jawab, jam operasional, dan keterangan singkat. Data yang tampil harus berasal dari layanan yang ditekan, bukan nilai tetap.
- Terapkan pengiriman nilai balik menggunakan `Navigator.pop(context, nilai)` pada tombol Ajukan Permohonan, kemudian tampilkan nilai tersebut pada halaman daftar layanan menggunakan `SnackBar`.

- Terapkan BottomAppBar beserta FloatingActionButton pada salah satu halaman penuh, misalnya halaman Riwayat Laporan yang dibuka dari tujuan Warga, dengan minimal tiga ikon aksi. NavigationBar utama tidak boleh dihapus.
- Terapkan navigasi adaptif menggunakan MediaQuery atau LayoutBuilder dengan ambang 600 piksel logis. Layar sempit menampilkan NavigationBar di bawah, sedangkan layar lebar menampilkan NavigationRail di sisi kiri dan menyembunyikan bilah bawah.
- Pastikan aplikasi tidak berhenti paksa ketika nama *route* salah tulis. Uji dengan memanggil satu nama *route* yang tidak terdaftar, kemudian pastikan halaman penanganan *route* tidak dikenal yang muncul.
- Jalankan aplikasi pada emulator atau perangkat fisik dan pada peramban Google Chrome, kemudian ambil tangkapan layar tampilan layar sempit dan tampilan layar lebar.

Luaran yang Dikumpulkan

- Kode program lengkap yang diunggah ke repositori Git dengan riwayat *commit* yang menggambarkan tahapan pengerjaan.
- Laporan dalam format PDF yang dikumpulkan melalui LMS.
- Tangkapan layar seluruh tujuan dan halaman, yaitu Beranda, Layanan beserta ketiga *tab*, Warga, *drawer* dalam keadaan terbuka, halaman rincian layanan, pemberitahuan hasil pengajuan, halaman dengan BottomAppBar, tampilan layar lebar dengan NavigationRail, dan halaman penanganan *route* tidak dikenal.
- Diagram atau bagan sederhana yang menggambarkan alur navigasi antarhalaman, disertai keterangan halaman mana yang menambah tumpukan *route* dan halaman mana yang hanya mengganti isi.
- Penjelasan singkat pada laporan mengenai alasan pemilihan NavigationBar untuk tujuan utama, NavigationDrawer untuk menu pendukung, *tab navigation* untuk pengelompokan isi, serta *named route* untuk perpindahan halaman.
- Berkas laporan diberi nama Modul2_NIM_NamaLengkap.pdf dan diunggah melalui LMS sesuai tenggat yang ditentukan.

Soal 2: Buat Kerangka Desain Tugas Akhir Saudara

Ketentuan Pengerjaan

- Tentukan tema dan nama aplikasi tugas besar, melanjutkan Tugas PABW. Tuliskan satu paragraf singkat berisi masalah yang diselesaikan dan penggunaanya.
- Buat proyek Flutter bernama `tubes_namaaplikasi_nim` dengan folder `lib/pages` dan `lib/navigation`.
- Terapkan minimal tiga dari enam materi: *named routes*, push dan pop dengan pengiriman data, NavigationBar, NavigationDrawer, *tab navigation*, dan NavigationRail adaptif.
- *Named routes* wajib dipakai. Seluruh perpindahan halaman melewati `app_routes.dart`.
- Pilih *widget* navigasi sesuai kebutuhan aplikasi.
- Isi halaman boleh sementara. Data boleh ditulis langsung di dalam kode.
- Sertakan `onUnknownRoute` agar aplikasi tidak berhenti paksa saat nama *route* salah tulis.

Luaran yang Dikumpulkan

- Kode program pada repositori Git, terpisah dari Soal 1.
- Tangkapan layar setiap halaman yang sudah dibuat.
- Laporan Modul2_Soal2_NIM_NamaLengkap.pdf diunggah melalui LMS.

H. Format Pengumpulan Tugas

Laporan disusun secara berurutan dengan sistematika berikut.

1. Halaman Sampul (Cover)

Memuat judul praktikum, nama mahasiswa, NIM, kelas, mata kuliah, nama dosen pengampu, program studi, serta tahun akademik. Gunakan logo institusi dan tata letak yang rapi.

2. Tujuan Praktikum

Menuliskan tujuan yang ingin dicapai pada praktikum ini dengan kalimat sendiri, bukan menyalin ulang tujuan pembelajaran pada modul.

3. Alat dan Bahan

Menyebutkan spesifikasi perangkat keras yang digunakan serta perangkat lunak beserta versinya, misalnya Flutter SDK, Dart SDK, Android Studio, Visual Studio Code, dan Git.

4. Langkah-Langkah Praktikum

Menguraikan tahapan pengerjaan secara runtut mulai dari pembuatan proyek, pembuatan *stateless widget*, *stateful widget*, penyusunan layout dan styling, hingga penggabungan pada `main.dart`. Setiap langkah disertai potongan kode yang relevan dan penjelasan singkat.

5. Hasil dan Dokumentasi Program

Menampilkan tangkapan layar hasil aplikasi yang berjalan, sekurang-kurangnya tampilan awal dan tampilan setelah terjadi perubahan *state*. Setiap gambar diberi nomor, keterangan gambar, dan penjelasan singkat.

6. Analisis dan Pembahasan

Membahas alasan pemilihan *stateless widget* dan *stateful widget* pada masing-masing bagian aplikasi, kendala atau galat yang ditemukan beserta cara penyelesaiannya, serta keterkaitan aplikasi dengan pilar smart city yang diangkat.

7. Kesimpulan

Menuliskan ringkasan capaian praktikum dan pemahaman yang diperoleh, disusun dalam beberapa kalimat yang menjawab tujuan praktikum.

8. Lampiran berisi Link Repositori GitHub

Mencantumkan tautan repositori GitHub yang dapat diakses oleh dosen pengampu, memuat kode program lengkap.

9. Presentasi Hasil Praktikum

Mahasiswa mempresentasikan hasil praktikum di kelas, meliputi demonstrasi aplikasi yang berjalan dan penjelasan kode program, serta menjawab pertanyaan dari dosen pengampu.